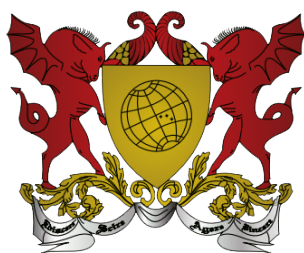




Linguagens de Programação para inteligência Artificial

Sabrina de Azevedo Silveira
Amanda Rossinol
Nicholas Costa



Universidade Federal de Viçosa

Reitor

Demetrius David da Silva

Vice-Reitora

Rejane Nascentes



Diretor

Francisco de Assis de Carvalho Pinto

Campus Universitário, 36570-900, Viçosa/MG

Telefone: (31) 3612 1251

Layout: Ana Luísa Medeiros e Stéfany Peron

Editoração Eletrônica: Stéfany Peron

Edição de conteúdo e CopyDesk: João Batista Mota



Significado dos ícones da apostila

Para facilitar o seu estudo e a compreensão imediata do conteúdo apresentado, ao longo de todas as apostilas, você vai encontrar essas pequenas figuras ao lado do texto. Elas têm o objetivo de chamar a sua atenção para determinados trechos do conteúdo, com uma função específica, como apresentamos a seguir.



DESTAQUE: são definições, conceitos ou afirmações importantes às quais você deve estar atento.



GLOSSÁRIO: Informações pertinente ao texto, para situá-lo melhor sobre determinado termo, autor, entidade, fato ou época, que você pode desconhecer.

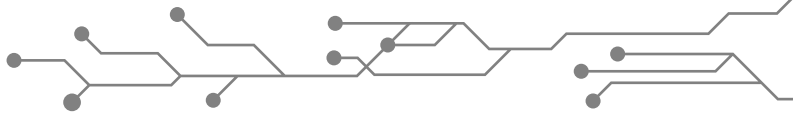
SAIBA MAIS: se você quiser complementar ou aprofundar o conteúdo apresentado na apostila, tem a opção de links na internet, onde pode obter vídeos, sites ou artigos relacionados ao tema.



PARA REFLETIR: vai fazer você relacionar um tópico a uma situação externa, em outro contexto



Exercícios Propostos: são momentos pra você colocar em prática o que foi aprendido

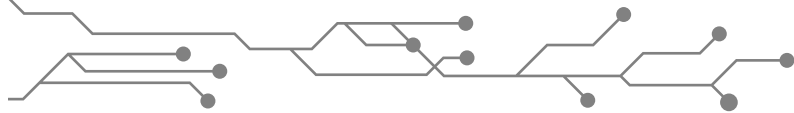


Apresentação

Esta é a apostila da disciplina *Linguagens de Programação para Inteligência Artificial*. Aqui, estão reunidos os conceitos elementares para o desenvolvimento de código nas linguagens Python, R e Octave. Inicialmente, apresentaremos os elementos básicos das linguagens, como variáveis, constantes, tipos de dados simples e indentação, seguidos pelos comandos básicos da linguagem e, por fim, serão abordados arranjos e funções.

Esses são os blocos básicos para implementação de programas em diversas linguagens de programação. Eles fornecem as ferramentas necessárias para que você possa começar a programar.

Bons estudos!



Autores

SABRINA DE AZEVEDO SILVEIRA

É graduada em Ciência da Computação pela Universidade Federal de Minas Gerais (UFMG-2008) e tem doutorado em Bioinformática (2013). Possui pós-doutorado no European Molecular Biology Laboratory-European Bioinformatics Institute (EMBL-EBI), em Cambridge, no Reino Unido (2019), e no Laboratório de Bioinformática e Sistemas (LBS), no Departamento de Ciência da Computação da Universidade Federal de Minas Gerais (2015).

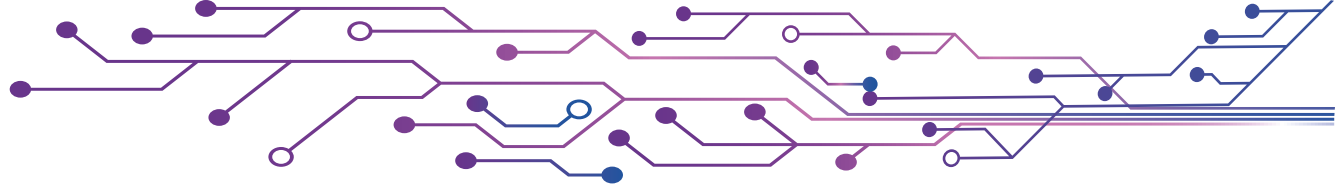
Atualmente é professora no Departamento de Informática (DPI), da Universidade Federal de Viçosa (UFV). É orientadora do Programa de Pós-Graduação em Ciência da Computação da UFV (CAPES 4) e do Programa de Pós-Graduação em Bioinformática da UFMG (CAPES 7).

Tem experiência na área de Ciência da Computação e Bioinformática, atuando principalmente nos seguintes temas: predição de função de enzima, mineração de dados, aprendizagem de máquinas, bases de dados biológicos e visualização de dados.

Contato: sabrina@ufv.br

AMANDA ROSSINOL E NICHOLAS COSTA

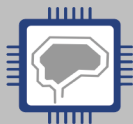
São estudantes da graduação em Ciência da Computação da UFV



Sumário

Apresentação

Python	7
	4
Capítulo 01 - Elementos básicos da linguagem Python	9
Capítulo 02 - Comandos básicos da linguagem Python	19
Capítulo 03 - Arranjos	35
Capítulo 04 - Funções	40
Capítulo 05 - Classes	46
06 - Exercícios	46
R	49
Capítulo 01 - Elementos básicos da linguagem R	50
Capítulo 02 - Comando Básicos	60
Capítulo 03 - Arranjos	66
Capítulo 04 - Funções	70
06 - Exercícios	73
Octave	75
Capítulo 01 - Elementos básicos da linguagem Octave	76
Capítulo 02 - Comandos básicos da linguagem Octave	82
Capítulo 03 - Arranjos	89
Capítulo 04 - Funções	94
05 - Exercícios	98



Python

**Editado por
Sabrina de A. Silveira**



Capítulo 01

Elementos básicos da linguagem Python

1. VARIÁVEIS, CONSTANTES E TIPOS DE DADOS SIMPLES

Nesta seção, veremos como utilizar variáveis para armazenar os dados em programas Python. Quando executamos o tão famoso programa *hello_world.py*, várias atividades estão sendo realizadas pelo Python.

```
1 print("Hello world!")
```

Listing 1: First program in Python.

A saída referente a esse código é:

```
1 Hello world
```

Ao executar o *hello_world.py*, a extensão *.py* indica que se trata de um programa em Python. O interpretador Python, então, percorre todo o programa e executa uma ação bem definida para cada palavra-chave da linguagem. Neste caso, quando o interpretador encontra a palavra-chave *print*, seguida de parênteses, ele imprime na tela o que estiver dentro de tais parênteses.

Agora, vamos adicionar uma variável a esse programa. Veja o Exemplo 2.

```
1 msg = "Hello world!"
2 print(msg)
```

Listing 2: Usando variável pela primeira vez.

O novo programa exibirá exatamente a mesma saída do anterior. A única diferença é que a mensagem de texto *"Hello world!"*, agora, está armazenada em uma variável chamada *msg*.



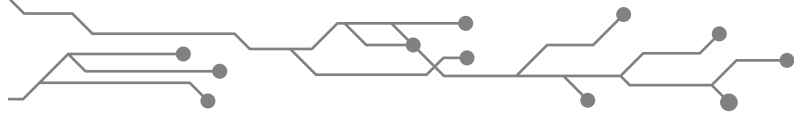
As variáveis são rótulos ou nomes que usamos dentro dos programas para nos referirmos a um espaço de memória RAM, no qual é armazenado algum dado. Apresentaremos, a seguir, como trabalhar com variáveis numéricas e depois com variáveis contendo texto.

2. NÚMEROS

O conceito de variável é semelhante ao da Matemática, mas um pouco diferente. Considere a expressão abaixo:

$$ax^2 + bx + c = 0$$

(1.1)



A equação 1.1, no contexto de Matemática, apresenta apenas 1 variável, x , enquanto os valores a , b , e c são tratados como constantes de uma equação de 2º grau. Na programação de computadores, x , a , b e c são vistos como variáveis, pois podem representar um valor numérico, e este pode variar, dependendo da equação que se deseja resolver.

Python trata os dados numéricos de diferentes formas, de acordo com o tipo. Começaremos pelos números inteiros.

2.1. Integers

Podemos adicionar (+), subtrair (-), multiplicar (*) e dividir (/) em Python, conforme o exemplo 3.

```
1 a = 2
2 b = 3
3 c = a + b
4 d = b - a
5 e = a * b
6 f = b / a
7 print("a + b = ", c)
8 print("b - a = ", d)
9 print("a * b = ", e)
10 print("b / a = ", f)
```

Listing 3: Operações básicas com inteiros.

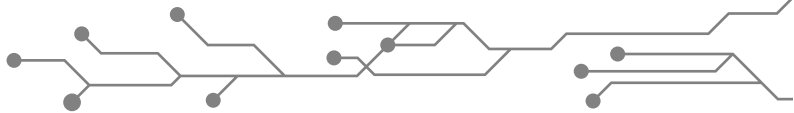
```
1 a + b = 5
2 b - a = 1
3 a * b = 6
4 b / a = 1.5
```

Podemos também realizar diversas outras operações, como exponenciação (exemplo 4).

```
1 a = 3
2 b = 2
3 c = a**b
4 d = 3**b
5 e = a**2
6 print(c, d, e)
```

Listing 4: Exponenciação com inteiros.

```
1 9 9 9
```



Múltiplas operações podem ser realizadas em uma expressão. Os parênteses podem ser utilizados para que as operações sejam executadas na ordem desejada. Veja o exemplo 5.

```
1 a = 2
2 b = 3
3 c = 4
4 r1 = a + b * c
5 print(r1)
6 r2 = (a + b) * c
7 print(r2)
```

Listing 5: Múltiplas operações em uma expressão.

```
1 r1 = 14
2 r2 = 20
```

O espaçamento não afeta a forma como o Python avalia as expressões. Ele é utilizado para melhorar a legibilidade do código.

2.2. Float

Os números de ponto decimal são chamados de *float* no contexto da linguagem de programação Python. Veja, no exemplo 6, a utilização desse tipo de dado.

```
1 a = 0.1
2 b = 0.2
3 c = a + b
4 d = a + a
5 e = 2 * b
6 print("c = ", c)
7 print("d = ", d)
8 print("e = ", e)
```

Listing 6: Uso de variáveis que representam números de ponto flutuante.

```
1 c = 0.30000000000000004
2 d = 0.2
3 e = 0.4
```

Observe que o resultado pode aparecer com um número arbitrário de casas decimais. Python procura representar o resultado da maneira mais precisa possível. Por ora, não se preocupe com isso.



Um ponto interessante é que, ao dividir dois números quaisquer, mesmo que sejam inteiros e que o resultado seja inteiro, Python representará esse resultado como *float*. Se números inteiros e de ponto flutuante forem utilizados em uma mesma operação, o resultado será do tipo *float*.

```
1 a = 4
2 b = 2
3 r1 = a/b
4 print("r1 = ", r1)
5 a = 1
6 b = 2.0
7 r2 = a + b
8 print("r2 = ", r2)
9 r3 = 2 * 3.0
10 print("r3 = ", r3)
11 r4 = 3.0 ** 2
12 print("r4 = ", r4)
```

Listing 7: Uso de variáveis que representam números de ponto flutuante.

```
1 r1 = 2.0
2 r2 = 3.0
3 r3 = 6.0
4 r4 = 9.0
```

3. CONSTANTES

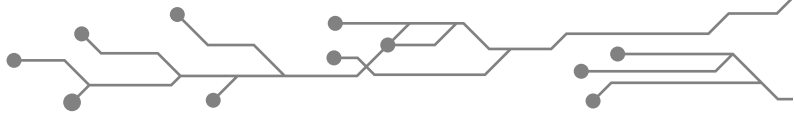
O conceito de constante é semelhante ao de variável. O que muda é que o valor de uma constante permanece o mesmo durante toda a execução de um programa. O Python não permite definir variáveis como constantes, porém, os programadores utilizam variáveis com nome em letras maiúsculas para indicar que a variável deve ser tratada como constante, não devendo ser alterada.

```
1 MAX_VALUE = 10
```

Listing 8: Uso de variáveis como se fossem constantes.

4. STRINGS

Os *strings* são uma série de caracteres. Qualquer conjunto de caracteres entre aspas (simples ou duplas) é considerado como um *string* em Python. Vejamos o exemplo 9.



```
1 mystring1 = "I have a son who is 6."
2 print(mystring1)
3 mystring2 = "ada lovelace"
4 print(mystring2)
```

Listing 9: Exemplos de variáveis do tipo string.

```
1 I have a son who is 6.
2 ada lovelace
```

A linguagem Python oferece grande flexibilidade para trabalhar com *strings*. Ilustraremos no exemplo 10 a utilização de variáveis dentro de um *string*, função e manipulação de espaços em branco.

```
1 # uso de funções
2 name = "ada lovelace"
3 print(name.title())
4 print(name.upper())
5 print(name.lower())
6 print()
7 first_name = "ada"
8 last_name = "lovelace"
9 # uso de f-strings
10 full_name = f"{first_name} {last_name}"
11 print(full_name)
12 print()
13 #removendo espaços em branco
14 favorite_language = ' python '
15 print(favorite_language.rstrip())
16 print(favorite_language.lstrip())
17 print(favorite_language.strip())
```

Listing 10: Exemplos de variáveis do tipo string.

```
1 Ada Lovelace
2 ADA LOVELACE
3 ada lovelace
4
5 ada lovelace
6
7 python
8 python
9 python
```


Das linhas 2 a 6 do exemplo 10, utilizamos métodos. Um método é alguma ação que o Python pode executar sobre um dado. O ponto (.) depois da variável *name* em *name.title()* indica ao Python que ele deve chamar o método *title()* para executar sua ação sobre a variável *name*. Como esse método não precisa de informações adicionais, os parênteses estão vazios. Há métodos que precisam de informações que serão passadas por meio de 1 ou mais variáveis. Nesse caso, informamos as variáveis, chamadas de argumentos, entre os parênteses.

Na linha 10 do exemplo 10 o *f* indica o uso do recurso de *f-strings*. Nesse caso, o Python formata o *string* trocando o nome das variáveis entre chaves pelo seu valor. Observe o resultado na linha 5 da saída.

Ainda no exemplo 10, nas linhas 15 a 17, são chamados métodos que fazem a remoção de espaços em branco à direita, à esquerda e ambos, respectivamente, no *string*. O resultado pode ser visto nas linhas 7 a 9 da saída. Os comandos *print()*, nas linhas 6 e 12, fazem uma quebra de linha.

5. OPERADORES ARITMÉTICOS

Já comentamos brevemente sobre as operações da Matemática que podemos realizar com o Python. Nesta seção, apresentamos os operadores aritméticos, que nos permitem realizar essas operações. Eles podem ser vistos na tabela 1.

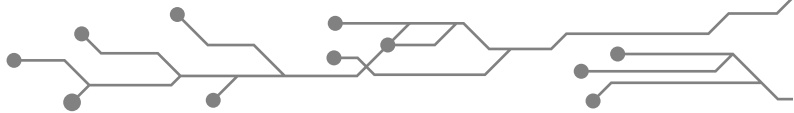
Table 1: Operadores aritméticos em Python

Operador	Descrição	Exemplo
()	Parênteses	(x + y) * 2
+	Adição	x + y
-	Subtração	x - y
*	Multiplicação	x * y
/	Divisão	x / y
%	Módulo (resto da divisão inteira)	x % y
//	Quociente (da divisão inteira)	x // y
**	Potenciação (x elevado a y)	x ** y

Para ilustrar, apresentamos no exemplo 11 um pequeno programa em Python que utiliza os principais operadores, acompanhado da saída que é gerada ao executar esse programa.

```
1 a = 9
2 b = 4
3
4 add = a + b
5 sub = a - b
6 mul = a * b
7 div1 = a / b
8 div2 = a // b
9 mod = a % b
10 p = a ** b
11
12 print("soma = ", add)
13 print("subtracao = ", sub)
14 print("multiplicacao = ", mul)
15 print("divisao = ", div1)
16 print("divisao_int = ", div2)
17 print("modulo = ", mod)
18 print("potenciacao = ", p)
```

Listing 11: Exemplos de uso dos operadores aritméticos. Adaptado de <https://www.geeksforgeeks.org/python-operators/>, acessado em janeiro de 2021.



Um ponto importante, quando falamos de operadores, é a sua precedência, ou seja, a prioridade que têm quando um programa em Python é executado. Na Tabela 2, é apresentada a precedência dos operadores aritméticos.

Table 2: Precedência de operadores aritméticos. A precedência começa pela maior, no topo da tabela e diminui até chegar à base da tabela.

Operadores	Nomes dos operadores
()	Parênteses
**	Potenciação
*, /, //, %	Multiplicação, Divisão, Módulo
+, -	Adição, Subtração

Apresentamos, na Tabela 3, alguns exemplos nos quais a precedência de operadores pode alterar significativamente os resultados. Essas operações podem ser testadas diretamente no Python shell.

Table 3: Operadores aritméticos e sua precedência.

Expressão	Resultado
8+3*2	14
(8+3)*2	22
2+4%3	3
(2+4)%3	0
5-3+1	3
5-(3+1)	1
1+4/2+2*2	7

6. OPERADORES RELACIONAIS

Os operadores relacionais são aqueles utilizados para fazer a comparação de valores. O resultado dessa comparação pode ser verdadeiro *True*, ou falso *False*.

Chamamos esses valores de booleanos ou *Boolean values*. Esse é mais um tipo de dado, de modo que uma variável em Python pode ser do tipo *bool*, e assim ela poderá assumir apenas os valores *True* ou *False*. A Tabela 4 apresenta operadores relacionais.

Table 4: Operadores relacionais

Operador	Descrição	Exemplo
>	Maior que: True se o operando à esquerda é maior que o à direita	<code>x > y</code>
<	Menor que: True se o operando à esquerda é menor que o à direita	<code>x < y</code>
==	Igualdade: True se ambos operandos são iguais	<code>x == y</code>
!=	Diferente: True se os operandos são diferentes	<code>x != y</code>
>=	Maior ou igual: True se operando à esquerda é maior ou igual ao à direita	<code>x >= y</code>
<=	Menor ou igual: True se operando à esquerda é menor ou igual ao operando à direita	<code>x <= y</code>

Apresentamos, no exemplo 12, um pequeno programa em Python que utiliza os principais operadores lógicos, acompanhado da saída, que é gerada ao executar esse programa.

```

1 a = 13
2 b = 33
3
4 print("a > b: ", a > b)
5 print("a < b:", a < b)
6 print("a == b:", a == b)
7 print("a != b:", a != b)
8 print("a >= b:", a >= b)
9 print("a <= b:", a <= b)

```

Listing 12: Uso de operadores relacionais. Adaptado de <https://www.geeksforgeeks.org/python-operators/>, acessado em janeiro de 2021.

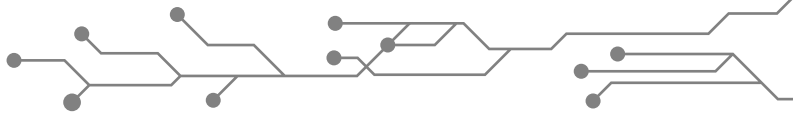
```

1 a > b: False
2 a < b: True
3 a == b: False
4 a != b: True
5 a >= b: False
6 a <= b: True

```

7. COMENTÁRIOS

Em linguagem de programação, os comentários são compostos por texto livre, que é utilizado para explicar um programa. São um recurso extremamente útil. Até o momento, tudo o que escrevemos nos programas é código Python. À medida que seus programas se tornam mais longos, é importante adicionar comentários para descrever a abordagem geral que você adotou para a resolução de um problema. É utilizado em partes específicas, como funções ou sequências de comandos, que possam demandar algum esforço para serem entendidos, para deixá-los mais legíveis.



Os comentários em Python são indicados por *hash mark* (#), conforme pode ser visto no programa 13. O interpretador Python irá ignorar linhas que iniciam com (#), elas são para melhorar a legibilidade do código para os programadores, algo muito importante se considerarmos que frequentemente os programas são desenvolvidos de modo colaborativo.

```
1 # Say hello to everyone.  
2 print("Hello Python people!")
```

Listing 13: Comentário em Python.

```
1 Hello Python people!
```

8. INDENTAÇÃO

A indentação se refere aos espaços que são colocados no início de uma linha de código. Em várias linguagens de programação, esses espaços são úteis para melhorar a legibilidade do código, mas, em Python, ela delimita um bloco de código e pode mudar o escopo de um comando, o que altera a semântica de um programa. Isso ficará mais claro quando falarmos dos comandos condicionais. Por ora, vejamos um exemplo de indentação no código 14.

```
1 if 5 > 2:  
2     print("Cinco é maior que 2!")
```

Listing 14: Indentação em Python.

```
1 Cinco é maior que 2!
```

No exemplo apresentado, a indentação indica que o comando *print* só é executado caso a expressão `if 5 > 2` seja avaliada como verdadeira. O PEP 8 (*Style Guide for Python Code*) recomenda a utilização de 4 espaços para indentação. Num mesmo bloco de código, devem ser utilizados o mesmo número de espaços para indentação, caso contrário o interpretador Python gerará uma mensagem de erro.

9. AGRADECIMENTOS

O conteúdo deste capítulo foi produzido baseado em Matthes (2019), w3schools (2021) e GeeksforGeeks (2021).

10. REFERÊNCIAS

GeeksforGeeks. 2021. *Python programming language*.
Matthes, Eric. 2019. *Python crash course: a hands-on, project-based introduction to programming*. no starch press.
w3schools. 2021. *Python tutorial*.



Capítulo 02

Comandos básicos da linguagem Python

1. ATRIBUIÇÃO

O comando de atribuição = foi utilizado por diversas vezes nos exemplos apresentados e serão usados durante todo este curso. Este comando atribui valor a uma variável. Mais especificamente, o resultado à direita da expressão é atribuído ao operando à esquerda da expressão. Aqui apresentamos uma tabela com os principais operadores, com exceção dos que trabalham bit a bit.

Table 1: Comandos de atribuição.

Operador	Descrição	Exemplo
=	Atribui o valor da expressão ao lado direito do operador ao operando da esquerda	x = y + z
+=	Soma os dois operandos e atribui ao da esquerda	a += b
-=	Subtrai os dois operandos e atribui ao da esquerda	a -= b
*=	Multiplica os dois operandos e atribui ao da esquerda	a *= b
/=	Divide os dois operandos e atribui ao da esquerda	a /= b
%=	Calcula a mod b e atribui ao da esquerda	a %= b
//=	Calcula a // b e atribui ao da esquerda	a //= b
=	Calcula ab e atribui ao da esquerda	a **= b

2. ENTRADA E SAÍDA PADRÃO

A função *input* permite receber algum texto digitado pelo usuário no teclado, que é o dispositivo de entrada padrão. A função *input* pausa o programa durante a execução e aguarda que o usuário digite algum texto e pressione a tecla *Enter*. Uma vez que o programa recebe a entrada do usuário, ela pode ser atribuída a uma variável, para que possa ser utilizada no programa. Um exemplo é apresentado no programa 15.

```
1 message = input("Diga alguma coisa e eu vou repetir: ")
2 print(message)
```

Listing 15: Comando input em Python.

```
1 Diga alguma coisa e eu vou repetir: Olá pessoal!
2 Olá pessoal!
```

A função *input* precisa de um único parâmetro, que é a mensagem de texto exibida para o usuário durante a execução do programa. Essa mensagem deve ser clara e de fácil compreensão

para que o usuário saiba exatamente o tipo de informação que ele deve fornecer ao programa. No caso do exemplo 15 essa mensagem é “Diga alguma coisa e eu vou repetir.”.



Quando a função *input* é utilizada, o interpretador Python assume que a entrada fornecida pelo usuário é do tipo *string*, de modo que se a entrada for um valor numérico inteiro ou de ponto flutuante, isso deve ser sinalizado explicitamente, como no exemplo 16.

```
1 idade = int (input('Qual sua idade? '))
2 altura = float (input('Sua altura (m)? '))
3 print('Idade:', idade )
4 print('Altura:', altura, 'm')
```

Listing 16: Comando input em Python.

```
1 Qual sua idade? 30
2 Sua altura (m)? 1.64
3 Idade: 30
4 Altura: 1.64 m
```

A função *print* permite imprimir mensagens na tela (que é a saída padrão no Python). A mensagem pode ser um *string* ou outro objeto que será convertido em *string* para ser impresso. Alguns parâmetros importantes dessa função são:

- *objeto(s)*: um ou mais objetos que serão impressos.
- *sep='separator'*: separador opcional. Define qual será o separador utilizado para imprimir os objetos.
- *end='end'* : terminador que será impresso ao final da impressão.

Um pequeno exemplo da utilização da função *sep='print'* é apresentado no programa 17.

```
1 print("banana")
2 print("banana", "maçã", "abacaxi")
3 print("banana", "maçã", "abacaxi", sep=":")
4 print("banana", "maçã", "abacaxi", sep=":", end="-")
```

Listing 17: Comando print em Python.

```
1 banana
2 banana maçã abacaxi
3 banana:maçã:abacaxi
4 banana:maçã:abacaxi-
```

3. ESTRUTURAS CONDICIONAIS

Os comandos condicionais nos permitem testar determinada condição e direcionar o fluxo de execução de um programa para o ponto desejado dentro do código, dependendo se a condição é falsa ou verdadeira. Os operadores relacionais são utilizados em conjunto com comandos condicionais. Vejamos, a seguir, o programa 18, no qual o comando *print* só é executado se a condição for verdadeira.

```
1 age = 19
2 if age >= 18:
3     print("O voto é obrigatório!")
```

Listing 18: Comando print em Python.

```
1 O voto é obrigatório!
```

É comum precisar executar determinado conjunto de comandos, somente se a condição for verdadeira, e outro conjunto de comandos, caso a condição testada seja falsa. Nesse caso, pode ser utilizado o comando *if-else*. No programa 19, vemos um exemplo que ilustra esta situação:

```
1 age = 17
2 if age >= 18:
3     print("Você tem a idade mínima para dirigir.")
4 else:
5     print("Você ainda não tem idade mínima para dirigir.")
```

Listing 19: Comando print em Python.

```
1 Você ainda não tem idade mínima para dirigir.
```

Há casos em que existem mais de duas possíveis situações e os comandos *if-elif-else* podem ser utilizados para avaliá-las. O interpretador Python executa apenas um bloco numa cadeia de comandos *if-elif-else*. Quando ele encontra um teste verdadeiro, executa o bloco de código imediatamente posterior e pula o resto dos testes condicionais. No programa 20, apresentamos um exemplo de uso da cadeia de comandos *if-elif-else*.

```

1 age = 19
2 if age < 16:
3     print("Você ainda não pode votar.")
4 elif age < 18:
5     print("Você já pode votar.")
6 elif age < 70:
7     print("O voto é obrigatório.")
8 else:
9     print("Você pode votar.")

```

Listing 20: Comando print em Python.

```

1 O voto é obrigatório.

```

É importante pontuar que podemos utilizar múltiplos blocos usando *elif*. Podemos também omitir o bloco *else*.

4. OPERADORES LÓGICOS

Os operadores lógicos permitem executar as operações lógicas AND, OR e NOT. São frequentemente utilizados com comandos condicionais. Na Tabela 2 apresentamos esses operadores.

Table 2: Operadores lógicos

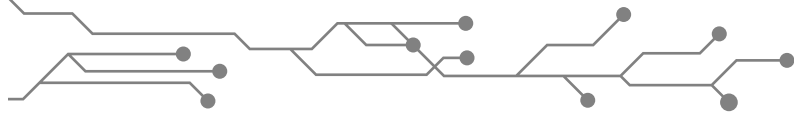
Operador	Descrição	Exemplo
and	AND: True se os 2 operandos são verdadeiros	x and y
or	OR: True se qualquer dos 2 operandos for verdadeiro	x or y
not	NOT: True se o operando for falso	not x

No caso do operador *and*, também chamado de (*e*), *x and y* resultará em verdadeiro somente se *a* e *b* forem verdadeiros. A tabela verdade para esse operador pode ser vista na Tabela 3.

Table 3: Operador and

x	y	x and y
True	True	True
True	False	False
False	True	False
False	False	False

O operador *or*, também chamado de (*ou*), resulta em verdadeiro se qualquer um dos 2 operandos for verdadeiro; ou seja, o resultado só será falso se os 2 operandos forem falsos, o que pode ser observado na Tabela 4.



O operador *not* é unário, o que significa que ele atua sobre um operando apenas. Ele inverte o valor do operando, conforme mostrado na Tabela 5.

A linguagem Python também tem os operadores lógicos *and*, *or* e *not*, que trabalham *bit a bit*, que são, respectivamente *&*, *|* e *~*. Eles estão fora do escopo deste curso.

Table 4: Operador or

x	y	x or y
True	True	True
True	False	True
False	True	True
False	False	False

Table 5: Operador not.

x	not x
True	False
False	True

Apresentamos, novamente, a precedência de operadores em Python na Tabela 6, agora incluindo os operadores relacionais e lógicos, além dos aritméticos, cuja precedência já havia sido apresentada.

Table 6: Precedência de operadores.

Precedência	Operadores e comandos
1	- (unário)
2	**
3	*, /, // %
4	+, -
5	<, <=, >, >=, == !=
6	not
7	and
8	or
9	= (atribuição)

No programa 21, há um exemplo da utilização em conjunto de comandos condicionais com operadores relacionais e lógicos. Nele, a nota do aluno é avaliada e o aluno pode ser considerado aprovado, reprovado ou ter direito a fazer prova final.

```
1 nota = float (input('Nota final: '))
2 if nota < 0 or nota > 100:
3     print('Nota informada é inválida.')
4 elif nota >= 60:
5     print('Aluno aprovado.')
6 elif nota >= 40:
7     print('Aluno tem direito a fazer prova final.')
8 else:
9     print('Aluno reprovado.')
```

Listing 21: Comandos condicionais, operadores relacionais e lógicos utilizados em conjunto.

```
1 Nota final: 101
2 Nota informada é inválida.
```

5. ESTRUTURAS DE REPETIÇÃO

As estruturas de repetição, também chamadas de *loop*, são muito úteis para que um bloco de instruções seja repetido diversas vezes, sem que seja necessário escrever tais instruções diversas vezes. Veremos as estruturas *while* e *for* nas próximas seções.

5.1 While

O *loop while*, ou melhor, as instruções dentro ou no escopo do *loop while* são executadas enquanto uma determinada condição é verdadeira. No programa 22, as instruções dentro do *loop while* são executadas 5 vezes, para contar de 1 a 5. Observe que é a indentação que determina as linhas que estão dentro do *while*.

```
1 current_number = 1
2 while current_number <= 5:
3     print(current_number)
4     # a linha abaixo é igual a current_number = current_number + 1
5     current_number += 1
```

Listing 22: Estrutura de repetição while.

```
1 1
2 2
3 3
4 4
5 5
```

Um programa também pode ser executado repetidas vezes até que o usuário digite um valor predefinido, que cause o fim da execução do programa. Essa situação é ilustrada no programa 23.

```
1 prompt = "\nDiga-me alguma coisa e eu vou repetir:"
2 prompt += "\nEntre 'pare' para terminar o programa. "
3 message = ""
4 while message != 'pare':
5     message = input(prompt)
6     print(message)
```

Listing 23: Estrutura de repetição while. O usuário decide quando o programa deve parar.

```
1 Diga-me alguma coisa e eu vou repetir:
2 Entre 'pare' para terminar o programa. Ola
3 Ola
4
5 Diga-me alguma coisa e eu vou repetir:
6 Entre 'pare' para terminar o programa. Ola novamente
7 Ola novamente
8
9 Diga-me alguma coisa e eu vou repetir:
10 Entre 'pare' para terminar o programa. pare
11 pare
```

Em situações em que há vários eventos diferentes que podem terminar a execução do programa, podemos criar uma variável que será avaliada para decidir se o programa deve continuar ou não. Esse tipo de variável é chamado de *flag*. Assim, o programa continua executando enquanto essa variável tem o valor *True*.

Caso aconteça algum evento que possa terminar a execução do programa, essa variável recebe o valor *False* e o programa termina. Um exemplo é disponibilizado no programa 24.

```
1 prompt = "\nDiga-me alguma coisa e eu vou repetir:"
2 prompt += "\nEntre 'pare' para terminar o programa. "
3 active = True
4 while active:
5     message = input(prompt)
6     if message == 'pare':
7         active = False
8     else:
9         print(message)
```

Listing 24: Estrutura de repetição while. O usuário decide quando o programa deve parar, agora com uma variável *flag*.

```

1 Diga-me alguma coisa e eu vou repetir:
2 Entre 'pare' para terminar o programa. Ola
3 Ola
4
5 Diga-me alguma coisa e eu vou repetir:
6 Entre 'pare' para terminar o programa. Ola de novo
7 Ola de novo
8
9 Diga-me alguma coisa e eu vou repetir:
10 Entre 'pare' para terminar o programa. pare

```

Outra alternativa para a utilização da estrutura de repetição *while* é a utilização do comando *break*. Este comando possibilita que o *loop* seja interrompido sem rodar nenhuma das instruções restantes dentro do *loop*. O programa 25 exemplifica o uso do *break*.

```

1 while True:
2     city = input(prompt)
3     if city == 'pare':
4         break
5     else:
6         print(f"Eu gostaria de visitar {city.title()}!")

```

Listing 25: Comando *while* utilizado em conjunto com *break*.

```

1 Informe o nome de uma cidade que você visitou:
2 (Digite 'pare' quando tiver informado a cidade.) Praga
3 Eu gostaria de visitar Praga!
4
5 Informe o nome de uma cidade que você visitou:
6 (Digite 'pare' quando tiver informado a cidade.) pare

```

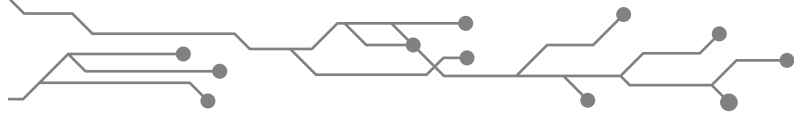
Ao invés de sair do *while* sem executar as demais instruções - usando para isso o comando *break* -, você também pode utilizar o comando *continue* para retornar ao início do *loop*, com base no resultado de um teste condicional. Um exemplo do uso do comando *continue* é apresentado no programa 26, que conta de 1 a 10 e imprime na tela os números ímpares.

```

1 current_number = 0
2 while current_number < 10:
3     current_number += 1
4     if current_number % 2 == 0:
5         continue
6     print(current_number)

```

Listing 26: Comando *while* utilizado em conjunto com *continue*.



```
1 1
2 3
3 5
4 7
5 9
```

É importante ter cuidado com os *loops* infinitos. Toda estrutura de repetição deve parar de executar em algum momento. Caso isso não aconteça, você deve estar em uma situação em que a condição de parada nunca é satisfeita.

5.2. For

A estrutura de repetição *for* é frequentemente utilizada para iterar (ou percorrer) uma sequência de elementos: pode ser um vetor, uma lista, um *string*, dentre outros.

No próximo capítulo, serão abordados arranjos e utilizados diversos exemplos de uso do *for*. Por ora, apresentaremos esse comando de maneira breve. No programa 27, mostramos o uso do comando para percorrer uma lista de frutas.

```
1 fruits = ["banana", "uva", "abacaxi", "melancia"]
2 for x in fruits:
3     print(x)
```

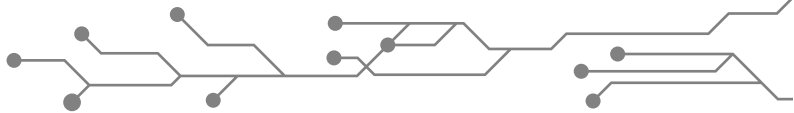
Listing 27: Comando *for* utilizado para percorrer uma lista.

```
1 banana
2 uva
3 abacaxi
4 melancia
```

O comando *for* também pode ser utilizado para percorrer ou iterar sobre os caracteres de um *string*, conforme mostrado no programa 28.

```
1 for x in "melancia":
2     print(x)
```

Listing 28: Comando *for* utilizado para percorrer os caracteres de um *string*.



```
1 m
2 e
3 l
4 a
5 n
6 c
7 i
8 a
```

De modo semelhante ao que vimos na estrutura de repetição *while*, podemos utilizar os comandos *break* e *continue* em conjunto com *for*. Para iterar determinado número de vezes com o comando *for*, podemos utilizar a função *range()*. O programa 29 mostra um exemplo utilizando os 2 comandos. Nele, o *loop* é repetido 5 vezes e o valor de *x* varia de 0 a 4.

```
1 for x in range(5):
2     print(x)
```

Listing 29: Comando *for* utilizado em conjunto com a função *range*.

```
1 0
2 1
3 2
4 3
5 4
```

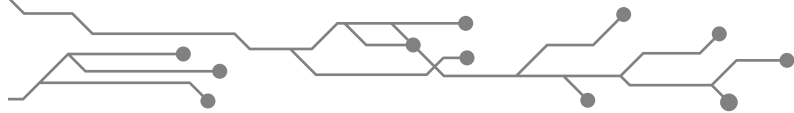
A função *range* também pode ser utilizada para especificar os valores de início e fim da repetição, conforme o exemplo 30.

```
1 for x in range(2, 6):
2     print(x)
```

Listing 30: Comando *for* utilizado em conjunto com a função *range*, informando ponto de início e fim do loop.

```
1 2
2 3
3 4
4 5
```

É possível ainda especificar o valor de início e fim da repetição, bem como o passo, que é a variação do valor a cada iteração. O exemplo 31 ilustra essa possibilidade. O valor inicial é 0, o valor final é 20 e o passo é 4.



```
1 for x in range(0, 21, 4):
2     print(x)
```

Listing 31: Comando for utilizado em conjunto com a função range, informando ponto de início, fim e passo do loop.

```
1 0
2 4
3 8
4 12
5 16
6 20
```

6. ENTRADA E SAÍDA A PARTIR DE ARQUIVOS

Nesta seção, apresentaremos a leitura e escrita de arquivos texto em Python. Não pretendemos esgotar o tema, pois estaria fora do escopo deste curso, mas sim apenas fornecer aos alunos uma noção básica de como abrir, fechar, ler e escrever em arquivos.

O primeiro passo para ler ou escrever num arquivo é abri-lo, utilizando a função *open*. Essa função aceita 6 modos de leitura:

- Somente leitura ('r')
- Leitura e escrita ('r+')
- Somente escrita ('w')
- Leitura e escrita ('w+')
- Somente acrescentar ('a')
- Acrescentar e ler ('a+') :

No caso das duas opções para leitura e escrita, a diferença é que ('r+') dá erro de entrada e saída, chamado de erro de I/O, se o arquivo não existir, enquanto em ('w+') o arquivo é sobrescrito se já existir.

A utilização da função *open* é ilustrada no programa 32. Neste exemplo, o arquivo deve estar no mesmo diretório de trabalho. A letra "r" é utilizada no segundo comando. Ela sinaliza que o *string* não tem caracteres especiais, como, por exemplo, \|. O "r" pode ser ignorado, caso o arquivo esteja no mesmo diretório de trabalho (que é o caso do exemplo).

```
1 file1 = open("Teste.txt", "a")
2
3 file2 = open(r"Teste.txt", "w+")
```

Listing 32: Utilização da função open.



É importante pontuar que todo arquivo aberto deve ser adequadamente fechado quando o programa termina de utilizá-lo. Isso é importante, pois libera a memória ocupada pelo arquivo.

O programa 33 mostra o uso do comando `close()`, utilizado para fechar arquivos.

```
1 file1 = open("Teste.txt", "a")
2 file1.close()
3
4 file2 = open(r"\Teste.txt", "w+")
5 file2.close()
```

Listing 33: Utilização da função `open`.

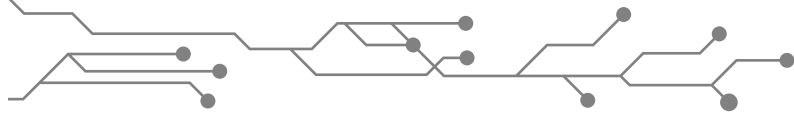
Veremos, a seguir, alguns exemplos de leitura de arquivos. No programa 34, o arquivo texto é lido de uma só vez com a função `read` e impresso na tela. Observe que a função `close` não é utilizada para fechar o arquivo. Isso acontece porque estamos utilizando o `with`, que já se encarrega de fechar o arquivo.

```
1 with open('caes.txt', 'r') as reader:
2     print(reader.read())
```

Listing 34: Utilização da função `read` para leitura de todo o arquivo.

```
1 Pug
2 Jack Russell Terrier
3 English Springer Spaniel
4 German Shepherd
5 Staffordshire Bull Terrier
6 Cavalier King Charles Spaniel
7 Golden Retriever
8 West Highland White Terrier
9 Boxer
10 Border Terrier
```

No programa 35, são lidos 5 *bytes* (5 caracteres) a cada chamada da função `readline`. Vai lendo de 5 em 5 até o fim da linha.



```
1 with open('caes.txt', 'r') as reader:
2     print(reader.readline(5))
3     print(reader.readline(5))
4     print(reader.readline(5))
5     print(reader.readline(5))
6     print(reader.readline(5))
```

Listing 35: Utilização da função readline para leitura.

```
1 Pug
2
3 Jack
4 Russe
5 ll Te
6 rrier
```

O arquivo pode ainda ser lido inteiro de uma só vez como uma lista utilizando a função *readlines()*. Dessa maneira, cada linha do arquivo será um elemento da lista. O exemplo 36 ilustra essa situação. Apenas um fragmento da saída desse programa foi exibido.

```
1 f = open('caes.txt')
2 list = f.readlines()
3 print(list)
```

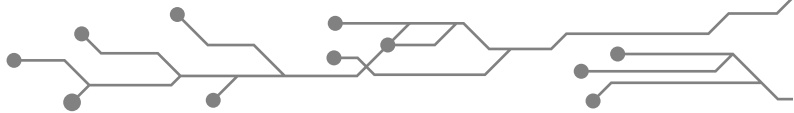
Listing 36: Utilização da função readlines para leitura de arquivo como lista.

```
1 ['Pug\n', 'Jack Russell Terrier\n', 'English Springer Spaniel\n', [...]]
```

No programa 37, um arquivo texto é percorrido linha por linha:

```
1 with open('caes.txt', 'r') as reader:
2     line = reader.readline()
3     # Enquanto há linhas para ler
4     while line != '':
5         print(line, end='')
6         line = reader.readline()
```

Listing 37: Lendo um arquivo linha por linha.



```
1 Pug
2 Jack Russell Terrier
3 English Springer Spaniel
4 German Shepherd
5 Staffordshire Bull Terrier
6 Cavalier King Charles Spaniel
7 Golden Retriever
8 West Highland White Terrier
9 Boxer
10 Border Terrier
```

No programa 38, um arquivo texto também é lido linha por linha, mas agora utilizando a função `readlines()`, que retorna uma lista e cada elemento dela representa uma linha do arquivo texto.

```
1 with open('caes.txt', 'r') as reader:
2     for line in reader.readlines():
3         print(line, end='')
```

Listing 38: Lendo um arquivo linha por linha, utilizando a função `readlines()`.

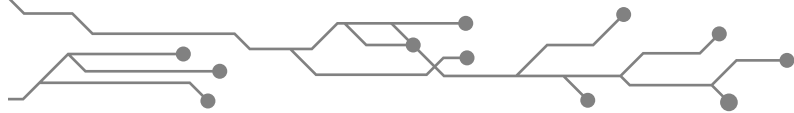
```
1 Pug
2 Jack Russell Terrier
3 English Springer Spaniel
4 German Shepherd
5 Staffordshire Bull Terrier
6 Cavalier King Charles Spaniel
7 Golden Retriever
8 West Highland White Terrier
9 Boxer
10 Border Terrier
```

No programa 39, são apresentadas duas alternativas para escrever em arquivo. Num primeiro momento, um arquivo é lido para uma lista (*dogs*), na qual cada elemento dessa lista é uma linha do arquivo.

Então, um segundo arquivo é escrito utilizando a função `writelines()`, que escreve os elementos de uma lista no arquivo de saída.

Um terceiro arquivo é gerado, percorrendo a lista *dogs* e escrevendo cada um de seus elementos como uma linha do arquivo de saída utilizando a função `write()`. A função `reversed()` inverte a ordem dos elementos da lista.

Apresentamos os arquivos de saída logo após o programa (como o conteúdo dos arquivos é o mesmo, apresentamos apenas 1).



```
1 with open('caes.txt', 'r') as reader:
2     # dogs é uma lista onde cada elemento é uma linha do arquivo
3     dogs = reader.readlines()
4
5 with open('caes_invertido.txt', 'w') as writer:
6     # escreve os elementos de uma lista com writelines
7     writer.writelines(reversed(dogs))
8
9 with open('caes_invertido2.txt', 'w') as writer:
10    # percorre elementos da lista dogs
11    for breed in reversed(dogs):
12        # escreve cada elemento (um string)
13        writer.write(breed)
```

Listing 39: Escrita em arquivo.

```
1 Border TerrierBoxer
2 West Highland White Terrier
3 Golden Retriever
4 Cavalier King Charles Spaniel
5 Staffordshire Bull Terrier
6 German Shepherd
7 English Springer Spaniel
8 Jack Russell Terrier
9 Pug
```

7. AGRADECIMENTOS

O conteúdo deste capítulo foi produzido baseado em Matthes (2019), w3schools (2021), GeeksforGeeks (2021) e RealPython (2021).

8. REFERÊNCIAS

GeeksforGeeks. 2021. *Python programming language*.

Matthes, Eric. 2019. *Python crash course: a hands-on, project-based introduction to programming*. no starch press.

RealPython. 2021. *Reading and writing files in python*.

w3schools. 2021. *Python tutorial*



Capítulo 03

Arranjos

Os arranjos são conjuntos de elementos de um mesmo tipo. Cada posição do arranjo é indexada por um número inteiro, começando a partir do 0. Dessa forma, um *array* é um conjunto de posições reservadas na memória do computador para armazenar esses elementos do mesmo tipo. Podemos imaginar essa estrutura como um diagrama, conforme a Figura 1.

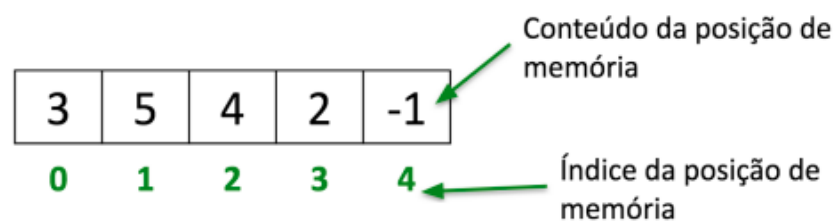


Figure 1: Vetor de tamanho 5. As posições são indexadas de 0 a 4.

O acesso ao elemento na quarta posição pode ser feito por meio da sintaxe `v[3]`. O conteúdo dessa posição é 2. O mesmo valor pode ser armazenado em diferentes posições no arranjo. Os arranjos podem ser de 1, 2, 3 ou *n* dimensões.

1. VETORES

Neste curso, utilizaremos a biblioteca *Numpy* do Python para criar e manipular arranjos numéricos. Com ela, é mais fácil criar e inicializar arranjos, além de ser mais semelhante a outras linguagens de programação. No programa 40, mostramos um exemplo em que essa biblioteca é utilizada para criar e inicializar vetores.

```
1 import numpy as np # np eh como chamamos o numpy para ficar sucinto
2
3 notas = np.empty( 5 ) # vetor de float, pode ter lixo na memoria
4 print("notas = ", notas)
5 notas2 = np.zeros( 5 ) # vetor de float inicializado com 0
6 print("notas2 = ",notas2)
7 idades = np.zeros( 5, dtype=int ) # vetor de int inicializado com 0
8 print("idades = ", idades)
9 contador = np.array([4, 3, 2, 1]) # inicializa na criaçao
10 print("contador = ", contador)
```

Listing 40: Criando e inicializando.

```
1 notas = [ 0.00000000e+000 -2.00000012e+000  2.17150342e-314
2 2.17151221e-314 2.13615776e-314]
3 notas2 = [0. 0. 0. 0. 0.]
4 idades = [0 0 0 0 0]
5 contador = [4 3 2 1]
```

Quando usamos arranjos, é muito comum precisarmos percorrê-los, realizando algum tipo de tarefa, como operações matemáticas e impressão formatada na tela, dentre outros. No programa 41, são apresentados alguns exemplos onde vetores são percorridos.

```
1 import numpy as np
2
3 n = int( input('Informe o número de elementos: '))
4 A = np.empty( n ) # cria arranjo
5
6 # le do teclado
7 for i in range(0, n):
8     A[i] = float( input('Informe o %dº elemento: ' % (i+1)))
9
10 # multiplica o vetor por 3
11 for i in range(0, n):
12     A[i] = A[i] * 3
13
14 # escreve na tela com formatacao
15 print("Imprimindo vetor na tela...")
16 for i in range(0, n):
17     print('%5.2f' % A[i], end='')
18 print()
```

Listing 41: Percorre vetor para ler do teclado, multiplicar por 3 e imprimir na tela.

```
1 Informe o número de elementos: 4
2 Informe o 1º elemento: 1
3 Informe o 2º elemento: 2.5
4 Informe o 3º elemento: 1
5 Informe o 4º elemento: 3
6 Imprimindo vetor na tela...
7 3.00 7.50 3.00 9.00
```

2. STRINGS

Os *strings* em Python são vetores de caracteres. Podemos acessar os elementos de *strings* utilizando colchetes, do mesmo modo que fizemos para os vetores. Podemos também percorrer os *strings* e executar diversas tarefas. No programa 42, ilustramos algumas delas.

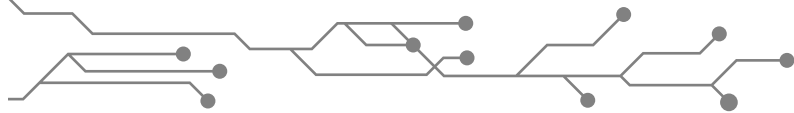
```
1 a = "Diga Olá!"
2 print("Caracter de índice 1...")
3 print(a[1]) # acessa elemento do string
4 print()
5
6 # percorre elemento do string 1 a 1
7 print("Imprime string...")
8 for x in a:
9     print(x, end="")
10 print("\n")
11
12 # percorre string com passo 2
13 print("Imprime string...")
14 for i in range(0, len(a), 2):
15     print(a[i], end="")
```

Listing 42: Iteração sobre strings.

```
1 Caracter de índice 1...
2 i
3
4 Imprime string...
5 Diga Olá!
6
7 Imprime string...
8 Dg l!
```

3. MATRIZES

As matrizes são arranjos de 2 dimensões. Podemos pensar nas matrizes como um diagrama com linhas e colunas. Para acessar um elemento, usamos a sintaxe `v[1][2]`, que indica que queremos acessar o elemento na linha de índice 1 e na coluna de índice 2, lembrando que os índices começam no 0. O elemento `v[1][2]` tem o valor 3 na matriz apresentada na Figura 2. Essa matriz tem 3 linhas e 4 colunas, com 12 elementos de ponto flutuante no total.



		colunas			
			↓		
	Índices	0	1	2	3
linhas →	0	4	1	2.5	2
	1	-1	0	3	-2
	2	0.2	6	5	-0.1

Os dados da matriz estão na memória do computador

Figure 2: Matriz de 3 linhas e 4 colunas.

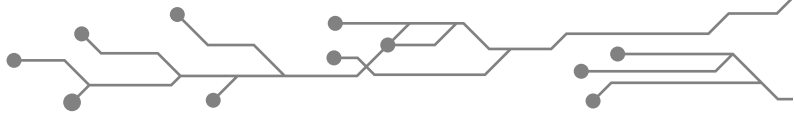
No programa 43, apresentamos um exemplo de leitura de matriz, elemento a elemento, e sua impressão na tela.

```

1  import numpy as np
2
3  # le dimensões da matriz
4  nl = int( input('Informe o número de linhas : '))
5  nc= int( input('Informe o número de colunas: '))
6
7  # cria matriz
8  A = np.empty( (nl, nc) )
9
10 # le do teclado
11 for i in range( 0, nl ): # linhas vao de 0 a nl-1
12     for j in range( 0, nc ): # colunas vao de 0 a nc-1
13         s = 'Informe o elemento A[%d][%d]: ' % (i, j)
14         A[i][j] = float (input( s ))
15
16 print("Imprime matriz...")
17 for i in range(0, nl):
18     for j in range(0, nc):
19         print('%5.1f' % (A[i][j]), end='')
20     print() # imprime quebra de linha
21 print()

```

Listing 43: Iteração sobre os elementos de matriz para leitura a partir do teclado e escrita na tela.



```
1 Informe o número de linhas : 3
2 Informe o número de colunas: 4
3 Informe o elemento A[0][0]: 4
4 Informe o elemento A[0][1]: 1
5 Informe o elemento A[0][2]: 2.5
6 Informe o elemento A[0][3]: 2
7 Informe o elemento A[1][0]: -1
8 Informe o elemento A[1][1]: 0
9 Informe o elemento A[1][2]: 3
10 Informe o elemento A[1][3]: -2
11 Informe o elemento A[2][0]: 0.2
12 Informe o elemento A[2][1]: 6
13 Informe o elemento A[2][2]: 5
14 Informe o elemento A[2][3]: -0.1
15 Imprime matriz...
16 4.0 1.0 2.5 2.0
17 -1.0 0.0 3.0 -2.0
18 0.2 6.0 5.0 -0.1
```

4. AGRADECIMENTOS

O conteúdo deste capítulo foi produzido baseado em Matthes (2019), w3schools (2021) e GeeksforGeeks (2021).

5. REFERÊNCIAS

GeeksforGeeks. 2021. *Python programming language*.

Matthes, Eric. 2019. *Python crash course: a hands-on, project-based introduction to programming*. no starch press.

w3schools. 2021. *Python tutorial*.



Capítulo 04

Funções

As funções são blocos de código que recebem um nome e executam determinada tarefa. Quando precisamos realizar a tarefa que foi definida numa função, não é preciso digitar tudo novamente, apenas chamamos a função. Para executar a tarefa, as funções podem precisar de dados, que chamamos de parâmetros ou argumentos. As funções também podem ou não retornar um ou mais valores ao final da execução.

1. DEFINIÇÃO DE FUNÇÕES

No programa 44, apresentamos a definição de uma função simples. A palavra chave *def* é usada para indicar ao Python que uma função está sendo definida. Em seguida, vemos o nome da função *greet_user()*.

Observe que temos parênteses vazios, o que indica que essa função não recebe parâmetros. A linha termina com *:*. As linhas indentadas que seguem a linha *def greet_user()*: representam o corpo da função. O corpo da função só é executado quando a função é chamada, e não quando ela é definida. A linha 3 apresenta um texto entre 3 aspas, chamado *docstring*, que descreve o que faz a função (esse recurso é útil para gerar documentação). A linha 4 contém o único comando que essa função executa.

Para usar a função, ou seja, para que o código dentro da função seja executado, é preciso chamar a função, o que foi feito na linha 8 do programa 44. Para isso, basta escrever o nome da função e passar os parâmetros necessários (neste exemplo, não há parâmetros).

```
1 # definição da função
2 def greet_user():
3     """Imprime saudação na tela"""
4     print("Olá!")
5
6 # chamada da função
7 print("Chamando a função greet_user()...")
8 greet_user()
```

Listing 44: Definição de função.

```
1 Chamando a função greet_user()...
2 Olá!
```

2. PASSAGEM DE PARÂMETRO

O programa 45 apresenta uma função que requer parâmetros. Além disso, mostra uma mesma função sendo chamada repetidas vezes, com parâmetros (dados) diferentes.





Listing 45: Definição de função com parâmetros.

3. RETORNO DE VALORES

Listing 46: Definição de função com parâmetros e valor de retorno.

cead^{UFV}

```
1 # definição da função
2 def ordena( x, y ):
3     if (x <= y):
4         return x, y
5     else:
6         return y, x
7
8 # programa principal
9 a = float( input('Digite um número: ') )
10 b = float( input('Digite outro número: ') )
11 a, b = ordena( a, b ) # chamada da função
12 print( a, '<=', b )
```

Listing 47: Definição de função com parâmetros e 2 valores de retorno.

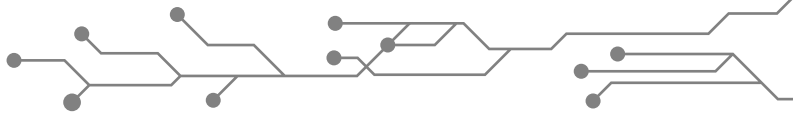
```
1 Digite um número: 7
2 Digite outro número: 4
3 4.0 <= 7.0
```

4. FUNÇÃO RECURSIVA

No programa 48, veremos uma função recursiva. Funções recursivas são aquelas que chamam a si mesmas. A função apresentada calcula a série de Fibonacci. Calcular a série sem **recursão** seria mais eficiente. O exemplo é apenas para ilustrar esse recurso.

```
1 # definição da função
2 def recursive_fibonacci(n):
3     if n <= 1:
4         return n
5     else:
6         # chamada recursiva
7         return (recursive_fibonacci(n - 1) + recursive_fibonacci(n - 2))
8
9 # programa principal
10 n_terms = 10
11 if n_terms <= 0:
12     print("Entrada inválida! Informe um valor positivo.")
13 else:
14     print("Serie de Fibonacci...")
15     for i in range(n_terms):
16         print(recursive_fibonacci(i), end=' ')
```

Listing 48: Cálculo da série de Fibonacci com recursão.



```
1 Serie de Fibonacci...
2 0 1 1 2 3 5 8 13 21 34
```

5. DEFINIÇÕES LOCAIS E GLOBAIS

Variáveis criadas fora de funções são chamadas variáveis globais. Elas podem ser usadas dentro e fora de funções. Se criamos dentro de uma função uma variável com o mesmo nome de outra que já existe fora da função, essa nova variável terá escopo local ao programa. A variável global de mesmo nome continuará global e com o valor original. O programa 49 ilustra essa situação.

```
1 # definição da função
2 def myfunc():
3     x = "fantástico"
4     print("Escopo local...")
5     print("Python é " + x)
6
7 # programa principal
8 x = "impressionante"
9 myfunc()
10 print("\nEscopo global...")
11 print("Python é " + x)
```

Listing 49: Variável local e global de mesmo nome.

```
1 Escopo local...
2 Python é fantástico
3
4 Escopo global...
5 Python é impressionante
```

Quando criamos uma variável dentro de uma função, ela é, em geral, local e, portanto, usada dentro da função. É possível criar uma variável global dentro de uma função usando a palavra chave *global*, o que pode ser visto no programa 50. Nesse programa, a variável *x* criada dentro da função *myfunc()* é global.

```
1 # definição da função
2 def myfunc():
3     global x
4     x = "fantástico"
5
6 # programa principal
7 myfunc()
8 print("Python é " + x)
```

Listing 50: Variável global definida dentro de função.

```
1 Python é fantástico
```

É possível mudar o valor de uma variável global dentro de uma função, utilizando a palavra chave *global*, conforme mostrado no programa 52. Em geral, é uma boa prática de programação evitar o uso de variáveis globais dentro de funções, devido ao risco de efeitos colaterais e perda da generalidade.

```
1 # definição da função
2 def myfunc():
3     global x
4     x = "fantástico"
5
6 # programa principal
7 x = "incrível"
8 myfunc()
9 print("Python é " + x)
```

Listing 51: Variável global alterada dentro de função.

```
1 Python é fantástico
```

6. AGRADECIMENTOS

O conteúdo deste capítulo foi produzido baseado em Matthes (2019), w3schools (2021) e GeeksforGeeks (2021).

7. REFERÊNCIAS

GeeksforGeeks. 2021. *Python programming language*.

Matthes, Eric. 2019. *Python crash course: a hands-on, project-based introduction to programming*. no starch press.

w3schools. 2021. *Python tutorial*



Capítulo 05

Classes

As classes, no contexto da programação orientada a objetos, representam coisas e situações do mundo real. Com base nas classes, criamos ou “instanciamos” objetos. Uma classe define um comportamento geral que uma categoria de objetos deve apresentar.

1. DEFINIÇÃO

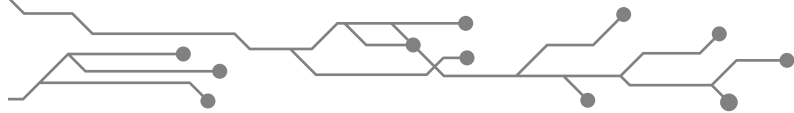
A classe *Dog* será criada para representar o animal cachorro de modo geral. Essa classe conterá dados sobre o cachorro, *name* e *age*, bem como comportamentos que ele pode apresentar, como *sit* e *roll_over*. O código 53 mostra a definição da classe *dog*. Essa classe foi armazenada dentro de um arquivo de nome *dog.py*.

```
1 class Dog:
2     """modelagem do animal cachorro"""
3
4     def __init__(self, name, age):
5         """inicializa os atributos name e age"""
6         self.name = name
7         self.age = age
8
9     def sit(self):
10        """Simula o cachorro atendendo ao comando sentar"""
11        print(f"{self.name} está sentado.")
12
13    def roll_over(self):
14        """Simula o cachorro atendendo ao comando rolar"""
15        print(f"{self.name} rolou!")
```

Listing 53: Definição de classe.

Por definição, o nome da classe começa com letra maiúscula em Python. Na linha 2, temos um *docstring*, que descreve a função da classe. As funções dentro da classe são chamadas de métodos. O método `__init__` é chamado automaticamente quando usamos a classe (o que veremos a seguir). Esse método, por convenção, deve ter 2 caracteres *underscore* no início e 2 no fim. Se o nome não obedecer a essa convenção, o método não é chamado automaticamente quando a classe é usada. O método `__init__(self, name, age)` apresenta 3 parâmetros. O parâmetro *self* é necessário e precisa ser o primeiro. Ele é uma referência para a própria instância da classe (quando a classe for instanciada, o que já veremos). Os parâmetros *name* e *age* precisam ser informados quando criamos uma instância da classe *Dog*.

As variáveis criadas nas linhas 6 e 7 são precedidas pela palavra *self* e são os atributos da classe. Essas variáveis podem ser acessadas por qualquer método da classe.



A classe tem ainda os métodos *sit* e *roll_over*, que não necessitam de atributos para funcionar e assim recebem apenas o parâmetro *self*. Por ora, esses métodos apenas imprimem na tela, mas num contexto real, como um jogo, por exemplo, poderiam conter comandos que fariam o objeto do tipo *Dog* sentar e rolar.

No programa 54, vamos utilizar a classe criada; ou seja, vamos criar uma instância do tipo abstrato de dados *Dog* para representar um cachorro específico. Observe que, na linha 2, importamos a classe *Dog* a partir do arquivo *dog*.

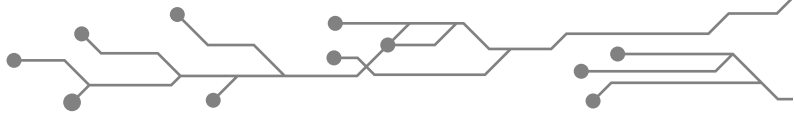
Outra alternativa seria importar todo o módulo, opção que deixamos comentada na linha 1. Na linha 4 é criado um objeto do tipo *Dog* com o nome *Willie* e idade 6. O Python utiliza o método *__init__(self, name, age)* para instanciar esse objeto. A instância criada é associada à variável *my_dog*. Para acessar os atributos do objeto, utilizamos *my_dog.name* e *my_dog.age*. Para chamar os métodos do objeto, utilizamos o nome da variável associada a ele seguido de *(.)* e do nome do método, conforme *my_dog.sit()* e *my_dog.roll_over()*. A saída do programa é apresentada logo na sequência.

```
1  #import dog
2  from dog import Dog
3
4  my_dog = Dog('Willie', 6)
5  print(f"O nome do meu cachorro é {my_dog.name}.")
6  print(f"Meu cachorro tem {my_dog.age} anos.")
7
8  my_dog.sit()
9  my_dog.roll_over()
```

Listing 54: Criando objeto a partir da classe.

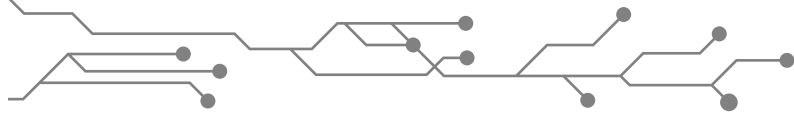
```
1  O nome do meu cachorro é Willie.
2  Meu cachorro tem 6 anos.
3  Willie está sentado.
4  Willie rolou!
```

Apresentaremos mais um exemplo de definição e utilização de classe. No código 55, fazemos a definição da classe *Car*. O programa 56 utiliza essa definição de classe para criar os objetos *my_used_car* e *my_new_car*. Nessa classe, criamos métodos para a leitura, *def read_odometer(self)*, e para a escrita, *def update_odometer(self, mileage)*, do atributo *odometer_reading*. Há também um método que faz o incremento desse atributo, *def increment_odometer(self, miles)*. Observe que, na linha 7, do programa 56, acessamos diretamente o atributo chamado *odometer_reading* e atribuímos a ele o valor 23. Uma boa prática de programação é definir métodos que façam a leitura e a atualização de atributos da classe, **evitando** que sejam acessados diretamente.



```
1 class Car:
2     """modelagem de carro através de classe"""
3     def __init__(self, make, model, year):
4         """inicializa os atributos que descrevem carro"""
5         self.make = make
6         self.model = model
7         self.year = year
8         self.odometer_reading = 0
9
10    def get_descriptive_name(self):
11        """Retorna um nome descritivo para carro."""
12        long_name = f"{self.year} {self.make} {self.model}"
13        return long_name.title()
14
15    def read_odometer(self):
16        """Imprime a distância percorrida pelo carro"""
17        print(f"Esse carro rodou {self.odometer_reading} mi.")
18
19    def update_odometer(self, mileage):
20        """Atualiza valor do odômetro para o valor fornecido."""
21        self.odometer_reading = mileage
22
23    def increment_odometer(self, miles):
24        """Incremente o odômetro com o valor fornecido."""
25        self.odometer_reading += miles
```

Listing 55: Definição da classe Car.



```
1 from car import Car
2
3 my_new_car = Car('audi', 'a4', 2019)
4 print(my_new_car.get_descriptive_name())
5 my_new_car.read_odometer()
6
7 my_new_car.odometer_reading = 23
8 my_new_car.read_odometer()
9
10 my_new_car.update_odometer(24)
11 my_new_car.read_odometer()
12 print()
13
14 my_used_car = Car('subaru', 'outback', 2015)
15 print(my_used_car.get_descriptive_name())
16 my_used_car.increment_odometer(10000)
17 my_used_car.read_odometer()
```

Listing 56: Definição da classe Car.

```
1 2019 Audi A4
2 Esse carro rodou 0 mi.
3 Esse carro rodou 23 mi.
4 Esse carro rodou 24 mi.
5
6 2015 Subaru Outback
7 Esse carro rodou 10000 mi.
```

2.AGRADECIMENTOS

O conteúdo deste capítulo foi produzido baseado em Matthes (2019).

3.REFERÊNCIAS

Matthes, Eric. 2019. *Python crash course: a hands-on, project-based introduction to programming*. no starch press.



06

Exercícios

1. ELEMENTOS BÁSICOS DA LINGUAGEM PYTHON

1) A fórmula para calcular a área de uma circunferência é: $\text{área} = \pi \cdot \text{raio}^2$. Considerando para este problema que π é constante e vale 3.14159.

2) Leia 2 valores de ponto flutuante de dupla precisão A e B, que correspondem a 2 notas de um aluno. A seguir, calcule a média dele, sabendo que a nota A tem peso 3.5 e a nota B tem peso 7.5 (A soma dos pesos, portanto, é 11). Assuma que cada nota pode ir de 0 até 10.0, sempre com uma casa decimal.

3) Leia 3 valores, no caso, variáveis A, B e C, que são as três notas de um aluno. A seguir, calcule a média desse estudante, sabendo que a nota A tem peso 2, a nota B tem peso 3 e a nota C tem peso 5. Considere que cada nota pode ir de 0 até 10.0, sempre com uma casa decimal.

4) Leia um número inteiro n maior ou igual à 100 e menor ou igual à 999. Imprima, separadamente, os dígitos que representam as unidades, dezenas e centenas do número.

5) Escreva um programa que leia o número de um funcionário, seu número de horas trabalhadas, o valor que recebe por hora e calcule o salário dele. A seguir, mostre o número e o salário do funcionário, com duas casas decimais.

6) Leia quatro valores inteiros A, B, C e D. A seguir, calcule e mostre a diferença do produto de A e B pelo produto de C e D segundo a fórmula: $\text{DIFERENCA} = (A * B - C * D)$.

1.1. Exercícios extras

<https://wiki.python.org.br/EstruturaSequencial>

2. COMANDOS BÁSICOS DA LINGUAGEM PYTHON

1) Faça um programa que imprima na tela os números de 1 a 20, um abaixo do outro. Depois modifique o programa para que ele mostre os números um ao lado do outro.

2) Faça um programa que leia um vetor de 10 números reais e mostre-os na ordem inversa.

3) Faça um programa que leia uma lista de 10 caracteres, e diga quantas consoantes foram lidas. Imprima as consoantes.

4) Faça um programa que leia a idade de 5 pessoas e imprima na tela quantas já atingiram a maioridade

5) Escreva um programa que receba 2 valores inteiros referentes à largura e altura de um retângulo e imprima na tela um retângulo com essas dimensões.

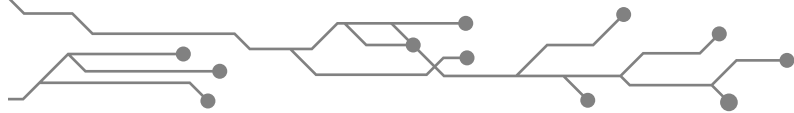
2.1 Exercícios extras

<https://wiki.python.org.br/EstruturaDeRepeticao>

<https://wiki.python.org.br/EstruturaDeDecisao>

3. ARRANJOS

1) Ordenar uma lista de valores pode ser muito útil e tem diversas aplicações na computação. Existem diversos algoritmos de ordenação, alguns mais simples, outros mais complexos, que envolvem conceitos não apresentados neste curso. O *bubble sort* é um algoritmo de ordenação



dos mais simples. A ideia é percorrer a lista diversas vezes e, a cada passagem, fazer flutuar para o topo o maior elemento da sequência. Dada uma lista de valores em ordem aleatória, use o algoritmo *bubble sort* para ordená-la e imprima o resultado.

2) Faça um programa que leia um vetor `X[10]`. Substitua, a seguir, todos os valores nulos e negativos do vetor `X` por 1. Em seguida, mostre o vetor `X`.

3) Faça um programa que leia 2 listas de tamanho 5 e crie uma terceira lista que tenha todos os elementos das duas listas.

4) Faça um programa que leia um valor `T` e preencha uma lista `N[20]` com a sequência de valores de 0 até `T-1` repetidas vezes, conforme exemplo: `T3 = 3`, `vector = [0, 1, 2, 0, 1, 2, 0, 1, 2, 0, 1, 2, 0, 1, 2, 0, 1, 2, 0, 1]`

5) Nome na vertical. Faça um programa que solicite o nome do usuário e imprima-o na vertical.

3.1 Exercícios extras

<https://wiki.python.org.br/ExerciciosComStrings>

<https://wiki.python.org.br/ExerciciosListas>

<https://wiki.python.org.br/ExerciciosArquivos>

4. FUNÇÕES

1) Faça um programa, com uma função que receba um argumento e retorne 'P', se o argumento for positivo, ou 'N', se o argumento for negativo

2) Faça uma função que retorne o fatorial de um número `N` maior que 0. Agora, refaça o problema proposto de modo recursivo.

3) Faça uma função que retorne a soma dos `N` primeiros números da sequência de Fibonacci.

4) Faça uma função que receba um vetor de números inteiros e retorne a soma dos elementos do vetor.

5) Faça uma função que receba um vetor de números inteiros e retorne quantos desses números são pares.

4.1 Exercícios extras

<https://wiki.python.org.br/ExerciciosFuncoes>

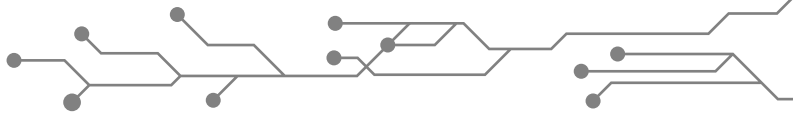
5. CLASSES

1) Crie uma classe chamada *Restaurant*. O método `__init__()` deve criar 2 atributos: *restaurant_name* e *cuisine_type*. Crie um método chamado *describe_restaurant()*, que imprime esses 2 atributos, e um método chamado *open_restaurant()*, que imprime uma mensagem indicando que o restaurante está aberto. Crie um objeto chamado *restaurante* a partir da classe definida. Imprima na tela os atributos individualmente e depois chame seus 2 métodos.

2) Utilize a classe do exercício anterior para criar 3 objetos e chame o método *describe_restaurant()* para cada um deles.

3) Defina uma classe chamada *User*. Crie 2 atributos chamados *first_name* e *last_name*. Crie também outros atributos que sejam comuns para um perfil de usuário. Defina um método chamado *describe_user()* que imprime um resumo da informação do usuário. Crie outro método chamado *greet_user()*, que imprime uma saudação personalizada para o usuário. Crie várias instâncias que representem diferentes usuários e chame os 2 métodos para cada um deles.

4) Utilize a classe *Restaurant* criada no exercício 1. Acrescente o atributo *number_served* com o valor 0. Crie uma instância dessa classe chamada *restaurant*. Imprima na tela o número de clientes que o restaurante serviu, altere esse valor e imprima novamente. Adicione o método *set_number_served()*, que permite que você atualize o número de clientes servidos. Chame o método passando um novo número e imprima o valor novamente. Crie o método *increment_*



number_served(increment) para incrementar o número de clientes servidos. Esse método, quando chamado, deve receber um valor que representa o incremento no número de clientes.

5) Adicione um atributo chamado *login_attempts* à classe *User* do exercício 3. Defina um método chamado *increment_login_attempts()* que incrementa em 1 unidade o atributo *login_attempts*. Defina agora o método *reset_login_attempts()*, que atribui o valor 0 ao mesmo atributo. Crie uma instância da classe *User* e chame o método *increment_login_attempts()* várias vezes. Imprima o valor de *login_attempts* para se certificar de que ele foi incrementado adequadamente. Finalmente, chame o método que atribui 0 a esse atributo, *reset_login_attempts()*. Imprima novamente *login_attempts* para se certificar de que o valor foi alterado para 0.

5.1 Exercícios extras

<https://wiki.python.org.br/ExerciciosClasses>

6. SUGESTÕES DE EXERCÍCIOS - URI ONLINE JUDGE

O URI Online Judge é um repositório de problemas de programação. Os problemas são separados por tópicos e por dificuldade, variando de 1 a 10. O portal oferece um sistema para que você possa testar suas soluções e saber na mesma hora se seu programa está correto ou não. É uma ótima ferramenta para treinar lógica de programação e fixar conceitos e palavras-chaves.

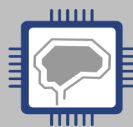
<https://www.urionlinejudge.com.br/judge/en/login>

7. AGRADECIMENTOS

O conteúdo deste capítulo foi produzido baseado em Matthes (2019).

8. REFERÊNCIAS

Matthes, Eric. 2019. Python crash course: a hands-on, project-based introduction to programming. no starch press.



R

**Editado por
Nicholas Costa
Sabrina de A. Silveira**



Capítulo 01

Elementos básicos da linguagem R

1. VARIÁVEIS

Uma variável representa um dado específico do programa que fica guardado em um espaço da memória física do computador - a memória RAM. São chamadas de variáveis, pois podem ter seus valores alterados durante a execução do programa.

R tem 5 tipos de dados básicos:

- Caractere
- Numérico (Números Reais)
- Inteiro
- Lógico
- Complexo

Sempre que uma variável for criada, deve-se nomeá-la de maneira eficiente; ou seja, seu nome será um resumo de sua funcionalidade. Essa estratégia faz com que o código fique mais claro e coerente. Logo após a escolha do nome, deve-se fazer a atribuição do valor. A linguagem R detecta automaticamente o tipo de dado atribuído à variável. Veja o exemplo 1:

```
msg <- "Hello world!"  
print(msg)
```

Listing 1: Usando variável pela primeira vez

Ao executar um trecho de código, o interpretador realiza exatamente os comandos que foram escritos. No exemplo anterior, o conteúdo guardado na variável *msg* será impresso.

```
Hello world!
```

Listing 2: Executando o primeiro programa

2. CONSTANTES

O conceito de constante é semelhante ao de variável. A diferença é que o valor de uma constante permanece o mesmo durante toda a execução de um programa. R não permite definir variáveis como constantes, porém, os programadores utilizam variáveis com nome em letras maiúsculas para indicar que uma variável deve ser tratada como constante, não devendo ser alterada. Um exemplo pode ser visto em 3.

```
MAX_VALUE = 10
```

Listing 3: Uso constante.

3. TIPOS DE DADOS

Como já foi dito, o conceito de variável é semelhante, mas um pouco diferente da Matemática. Considere a expressão abaixo:

$$ax^2 + bx + c = 0$$

(1.1)



A equação 1.1, no contexto de Matemática, tem apenas 1 variável, x , enquanto os valores a , b , e c são tratados como constantes de uma equação de 2º grau. Na programação de computadores, x , a , b e c são vistos como variáveis, pois podem representar um valor numérico, e este pode variar dependendo da equação que se deseja resolver.

R tem cinco tipos de dados que podem ser armazenados em suas variáveis, que são tratados de formas distintas.

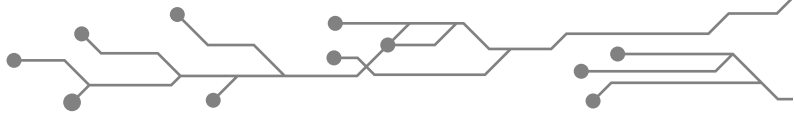
3.1. Numeric

O tipo Numeric, em R, corresponde aos números Reais. Podemos adicionar (+), subtrair (-), multiplicar (*) e dividir (/), conforme o trecho 4.

```
a <- 2
b <- 3
soma <- a + b
subtracao <- b - a
multiplicacao <- a * b
divi <- b / a
restoDiv <- b %% a      #operador que retorna o resto da divisão
parteIntDiv <- b %/% a   #operador que retorna a parte inteira da divisão

print(soma)
print(subtracao)
print(multiplicacao)
print(divi)
print(restoDiv)
print(parteIntDiv)
```

Listing 4: Operações básicas com tipo Numeric



```
5
1
6
1.5
2
0
```

Listing 5: Resultado das operações básicas

Outra operação que R permite é a exponenciação. Veja o trecho 6.

```
a <- 3
b <- 2
c <- a**b
d <- 3**b
e <- a**2

print(c)
print(d)
print(e)
```

Listing 6: Exponenciação com inteiros

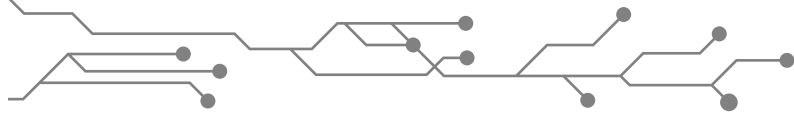
```
9
9
9
```

Listing 7: Resultado da exponenciação

Em uma única expressão, é possível realizar várias operações. Veja o exemplo 8.

```
a <- 2
b <- 3
c <- 4
r1 <- a + b * c
print(r1)
r2 <- (a + b) * c
print(r2)
```

Listing 8: Múltiplas operações em uma expressão



```
r1 <- 14
r2 <- 20
```

Listing 9: Resultado das expressões

3.2. Integer

O tipo Integer, em R, corresponde aos conjuntos dos números inteiros na Matemática. Eles são representados de uma forma diferente do tipo Numeric. Veja o trecho 10:

```
a <- 5L #é preciso colocar o L após o número para sinalizar que é do tipo Integer
b <- 3L
c <- a + b
d <- a + a
e <- 2L * b
print (c)
print (d)
print (e)
```

Listing 10: Variáveis inteiras

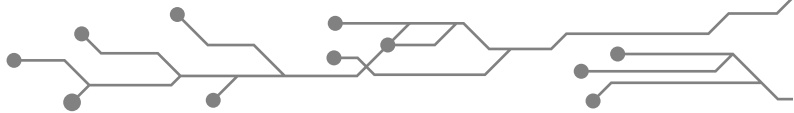
```
8
5
6
```

Listing 11: Resultado das operações com números inteiros

É importante pontuar que operar tipos de dados inteiros pode resultar na mudança do tipo. Veja o trecho 12:

```
a <- 5L
b <- 3L
c <- a / b    #operação sobre Tipos Integer que resultam em um tipo Numeric
d <- a * a    #permanece Integer
e <- 2 * b    #operação entre um tipo Integer e Numeric, o resultado será Numeric
class(c)      #imprime o tipo da variável
class(d) 8 class(e)
```

Listing 12: Variáveis inteiras



```
"numeric"  
"integer"  
"numeric"
```

Listing 13: Mudança de tipos de variáveis através de operações básicas

3.3. Complex

Representa um número pertencente ao conjunto dos números complexos. A parte complexa corresponde ao número seguido da letra i no final.

O exemplo 15 mostra o uso de números complexos em R.

```
a <- 4 + 5i  
b <- 6 + 2i  
c <- a - b  
  
print(c)  
class(c)
```

Listing 14: Tipos Complex

```
-2 + 3i  
"complex"
```

Listing 15: Tipos Complex

3.4. Logic

Os Operadores do tipo Logic são capazes de armazenar apenas dois valores: TRUE ou FALSE. Apesar de serem simples, são de extrema importância e muito úteis. Veja o trecho 16.

```
operador1 <- TRUE  
operador2 <- FALSE  
  
print(operador1)  
print(operador2)
```

Listing 16: Variáveis do tipo Logic

```
"logical"
"logical"
```

Listing 17: Variáveis do tipo Logic

3.5. Character

Uma série de caracteres. Qualquer conjunto de caracteres entre aspas (simples ou duplas), também conhecido como *string*. Vejamos o exemplo 18.

```
mystring1 <- "I have a son who is 6."
print(mystring1)
mystring2 <- "ada lovelace"
print(mystring2)
```

Listing 18: Exemplos de variáveis do tipo Character

```
I have a son who is 6.
ada lovelace
```

Listing 19: Exemplos de variáveis do tipo Character

4. OPERADORES E EXPRESSÕES ARITMÉTICAS

Já comentamos brevemente sobre as operações da Matemática que podemos realizar em R. Nesta seção, apresentamos os operadores aritméticos, que nos permitem realizar essas operações. Eles podem ser vistos na tabela 1.

Para ilustrar, apresentamos no trecho 20, um pequeno programa em R que utiliza os principais operadores, acompanhado da saída que é gerada ao executar esse programa.

operador	descrição	exemplo
()	Parênteses	(x + y) * 2
+	Adição	x+y
-	Subtração	x-y
*	Multiplicação	x*y
/	Divisão	x/y
	Módulo (resto da divisão inteira)	x %% y
%%	Quociente (da divisão inteira)	x %/% y
**	Potenciação (x elevado a y)	x ** y

Tabela 1: Operadores aritméticos em R

```

a <- 9
b <- 4
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a %% b)
print(a %/% b)
print(a ** b)

```

Listing 20: Exemplos de uso dos operadores aritméticos

```

13
5
36
2.25
2
1
6561

```

Listing 21: Operadores aritméticos

4.1. Precedência dos Operadores Aritméticos

Um ponto importante, quando falamos de operadores, é a sua precedência, ou seja, a prioridade que têm quando um programa em R é executado. Na Tabela 2, é apresentada a precedência dos operadores aritméticos.

operador	nome dos operadores
()	Parênteses
**	Potenciação
*, /, %%, %/%	Multiplicação, Divisão, Módulo, Quociente
+, -	Adição, Subtração

Tabela 2: Precedência de operadores aritméticos. A precedência começa pela maior, no topo da tabela, e diminui até chegar à base da tabela

Apresentamos, na Tabela 3, alguns exemplos nos quais a precedência de operadores pode alterar significativamente os resultados. Em caso de empates de precedência, o operador mais à esquerda será executado primeiro.

expressão	resultado
8+3*2	14
(8+3)*2	22
2+4%%3	3
(2+4)%3	0
5-3+1	3
5-(3+1)	1
1+4/2+2*2	4

Tabela 3: Operadores aritméticos e sua precedência

5. OPERADORES RELACIONAIS

Os operadores relacionais são aqueles utilizados para fazer a comparação de valores. O resultado dessa comparação pode ser verdadeiro (TRUE) ou falso (FALSE). Chamamos esses valores de booleanos ou Logical, em R. A Tabela 4 apresenta operadores relacionais.

operador	descrição	exemplo
>	Maior que: True se o operando à esquerda é maior que à direita	x>y
<	Menor que: True se o operando à esquerda é menor que à direita	x<y
==	Igualdade: True se ambos operandos são iguais	x==y
!=	Diferente: True se os operandos são diferentes	x!=y
>=	Maior ou igual: True se operando à esquerda é maior ou igual à direita	x>=y
<=	Menor ou igual: True se operando à esquerda é menor ou igual ao operando à direita	x<=y

Tabela 4: Operadores relacionais

Apresentamos, no trecho 22, um pequeno programa em R que utiliza os principais operadores relacionais, acompanhado da saída, que é gerada ao executar esse programa.

```
a <- 13
b <- 33

print(a > b)
print(a < b)
print(a == b)
print(a != b)
print(a >= b)
print(a <= b)
```

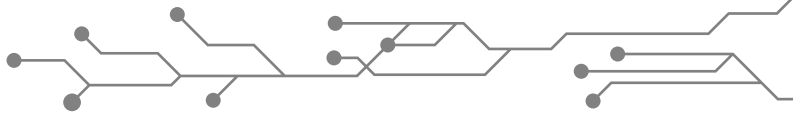
Listing 22: Uso de operadores relacionais

```
False
True
False
True
False
True
```

Listing 22: Uso de operadores relacionais

6. OPERADORES E EXPRESSÕES LÓGICAS

Além dos operadores aritméticos e relacionais, R conta com os operadores lógicos. As operações lógicas são feitas entre os valores booleanos. Dessa forma, é possível perceber que apenas dois resultados são possíveis: TRUE ou FALSE. A tabela 5 mostra os operadores lógicos disponíveis em R.



operador	descrição	exemplo
&	And: TRUE, se ambas condições são verdadeiras.	x&y
	Or: TRUE, se pelo menos uma das condições for verdadeira.	x y
!	Not: Inverte o valor lógico	!(x y)

Tabela 5: Operadores lógicos em R

Veja um exemplo sobre o uso dos operadores lógicos no trecho 24:
É importante mencionar que todos os operadores lógicos têm a mesma precedência e que, dessa forma, o desempate é dando prioridade para o operador mais à esquerda. O uso dos parênteses ajuda no entendimento das expressões.

```
a <- 5 > 6      #falso
b <- 6 != 100   #verdade
c <- -95 < 1    #verdade
print(a & b)    #uma das condições é falsa, retorna FALSE
print(a | c)    #uma das condições é verdadeira, retorna TRUE
print(!(a & b)) #inverte o valor lógico
```

Listing 24: Operadores lógicos

```
FALSE
TRUE
TRUE
```

Listing 25: Operadores lógicos

7. COMENTÁRIOS

Os comentários em linguagem de programação são compostos por texto livre, que é utilizado para explicar um programa. São um recurso extremamente útil. À medida que seus programas se tornam mais longos, é importante adicionar comentários que descrevem a abordagem geral que você adotou para a resolução de um problema.

Podem ser utilizados em partes específicas, como funções ou sequências de comandos que demandam algum esforço para serem entendidos, para deixá-los mais legíveis.

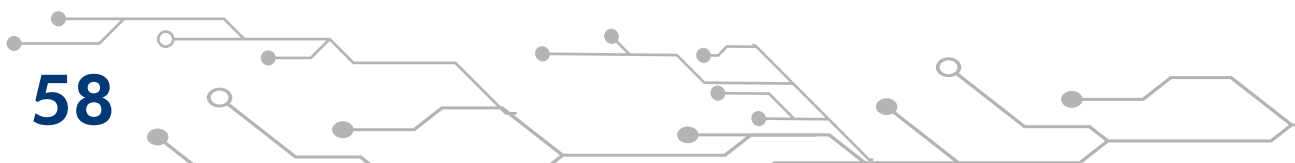
Os comentários em R são indicados por *hash mark* (**#**), conforme foi visto em alguns exemplos anteriores. O interpretador R irá ignorar linhas que iniciam com (**#**), elas são para melhorar a legibilidade do código para os programadores – isso é algo muito importante, se considerarmos que frequentemente os programas são desenvolvidos de modo colaborativo. Em 26, vemos um exemplo de comentário.

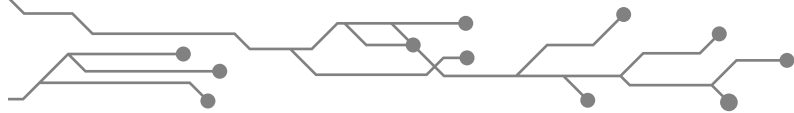
```
1 # Say hello to everyone.
2 print("Hello people!")
```

Listing 26: Comentário em R

```
Hello Python people!
```

Listing 26: Comentário em R





8. INDENTAÇÃO

A indentação se refere aos espaços que são colocados no início de uma linha de código. Em R, esses espaços são úteis para melhorar a legibilidade do código.

9.AGRADECIMENTOS

O conteúdo deste capítulo foi produzido baseado em Curso-R (2021) e em Eduardo Ferreira (2021).

10.REFERÊNCIAS

Curso-R, Material -. 2021. R básico.

Eduardo Ferreira, Weverton Lisboa. 2021. Introdução ao R.



Capítulo 02

Comando Básicos

1. COMANDOS BÁSICOS

Os comandos básicos são aqueles mais usados ao se programar e que estão presentes em códigos de qualquer nível de complexidade.

1.1. Atribuição de Valores

Em suma, a atribuição de valores consiste em gravar um dado em uma variável. Veja mais um exemplo no trecho 28:

```
a <- 5  
b <- "valor"
```

Listing 28: Atribuição de valores

A notação de atribuição não é nenhuma novidade, mas é importante destacar que, em R, existem dois operadores que têm a função de atribuir valores a uma variável. São eles: `<-` e `=`. Veja o resumo na tabela 1 (o conceito de função será visto mais à frente):

operador	descrição	exemplo
<code><-</code>	Atribuição. Usado por padrão para atribuir valores a variáveis	<code>x <- 3</code>
<code>=</code>	Atribuição. Atribuir argumentos a uma função	<code>f(5, x=3)</code>

Tabela 1: Operadores de atribuição em R

1.2 Entrada e Saída Padrão

1.2.1 Entrada

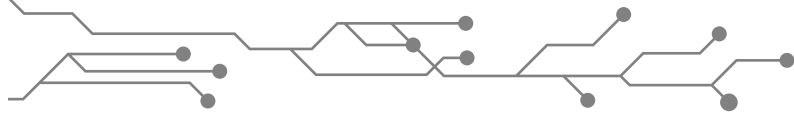
A entrada de dados em um código é quando o usuário escreve os dados que pretende usar durante a execução do programa. Essa ação é feita graças à função `scan()`. Após a sua execução, o programa espera que o usuário digite algum dado para poder prosseguir. Veja o exemplo no trecho 29:

```
scan()  
a <- 15  
a <- scan()  
  
print(a)
```

Listing 29: Entrada de dados via teclado

```
100 #valor passado via teclado, pelo o usuário
```

Listing 30: Entrada de dados via teclado



Observe que, no exemplo anterior, a função `scan()` foi chamada duas vezes, mas apenas o dado que foi atribuído à variável pôde ser acessado. A primeira chamada da função `scan()` não salva o valor recebido.

1.3 Saída

A função `print()` é utilizada para a saída de dados; ou seja, ela imprime na tela o dado que foi armazenado na variável. Observe o trecho 31:

```
variavel <- 15

print(variavel)
print("variavel")
```

Listing 31: Exemplos de saída

```
15
variavel
```

Listing 32: Exemplos de saída

1.4. Entrada e Saída a partir de arquivos

Se os dados já estão disponíveis em outro ambiente, isto é, já foram digitados em outros programas, é possível importá-los para o *script* em R e evitar reescrevê-los. Por exemplo, se os dados estão disponíveis em uma planilha do excel, basta salvá-los como `.csv` ou `.txt` e importar para o código.

1.4.1 Entrada

Para importar um arquivo, é preciso fazer isso explicitamente. O primeiro passo é mostrar o caminho do arquivo que deseja importar, usando a função `setwd()`. O segundo passo é ler o arquivo que deseja importar e gravá-lo em alguma variável com a função `read`. Veja o trecho 33.

```
1 setwd("home/R/arquivos")
#chega no local do arquivo
2 dado <- read.csv("arq.csv") #importa o arquivo
```

Listing 33: Exemplos de entrada com arquivo de texto

No exemplo 33, uma tabela criada no excel foi salva com a extensão `arq.csv`. Após gravar o arquivo importado em uma variável, é possível fazer manipulações por meio de outras funções que serão vistas mais à frente.

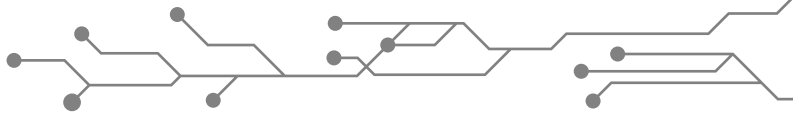
1.4.2. Saída

Em R, também é possível exportar dados. A função `write.table()` é capaz de criar um arquivo no formato `.csv` ou `.txt` com as especificações definidas pelos argumentos. Veja o exemplo no trecho 34:

```
write.table(dado, file = "home/R/arquivos/export.csv")
```

Listing 34: Exemplos de saída com arquivo de texto

No exemplo 34, os dados que estavam armazenados na variável `dado` foram exportados como um arquivo `.csv` para o caminho estipulado no argumento da função `write.table()`.



1.5. Estruturas Condicionais

1.5.1. If

Uma estrutura condicional executa um determinado trecho de código, se sua condição for verdadeira. Os operadores relacionais são utilizados em conjunto com as estruturas condicionais. Veja o trecho 35.

```
primeiraCondicao <- TRUE
segundaCondicao <- FALSE

if(primeiraCondicao){
  print("primeira condição aceita")
}

if(segundaCondicao){
  print("segunda condição aceita")
}
```

Listing 35: Exemplos de IF

```
primeira condição aceita
```

Listing 36: Exemplos IF

Perceba que apenas um dos *prints* foi executado. Isso acontece devido às condicionais, que executam determinado trecho de código, apenas se sua condição for satisfeita. No primeiro *if*, a condição é uma variável que tem o valor lógico TRUE; ou seja, sua condição é verdadeira. Já no segundo *if*, a condição é FALSE. Por isso, não houve execução.

1.5.2 If Else

O *else* é uma extensão do *if*. Confira o trecho 37:

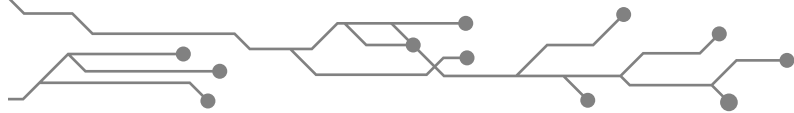
```
primeiraCondicao <- TRUE
segundaCondicao <- FALSE

#podemos ler essa estrutura da seguinte forma:
#se primeira condição for aceita, faça X
#se não, faça Y

if(primeiraCondicao){
  print("primeira condição aceita")
} else{
  print("primeira condição não aceita")
}

if(segundaCondicao){
  print("segunda condição aceita")
} else{
  print("segunda condição não aceita")
}
```

Listing 37: Exemplos de *if else*



primeira condição aceita
segunda condição não aceita

Listing 38: Exemplos de *if else*

Observe que o primeiro *if* é satisfeito, pois tem um argumento verdadeiro. Não verificado o *else*, pois o interpretador já encontrou a condição válida. No segundo *if*, a condição é falsa. Portanto, a parte executada será o *else*.

1.5.3 IFs alinhados

Também existe a possibilidade de IFs alinhados. Ou seja, uma maneira de se testar várias condições. Veja o exemplo do trecho 39.

```
if(5 > 3){  
  print("5 > 3")  
} else if( 6 != 6){  
  print("6 != 6")  
} else if( 14 <= 14){  
  print("14 <= 14")  
} else{  
  print("nenhuma opção é válida")  
}
```

Listing 39: Exemplos de IFs alinhados

5 > 3

Listing 40: Exemplos de IFs alinhados

No exemplo 40, há várias condições a serem testadas. A condição (5 > 3) é verdadeira, portanto, o interpretador executa seu trecho de código. É importante perceber que, após uma condição ter sido satisfeita, as condições subsequentes não serão testadas.

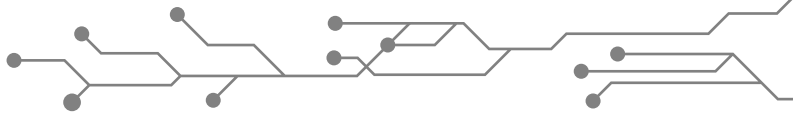
Outro ponto a se destacar é que os argumentos das estruturas condicionais são expressões lógicas. Dessa forma, é possível criar condições relativamente complexas. Observe no trecho 41:

```
a <- 14  
b <- 10  
c <- -1  
  
if( ((a > b) | (c != 0)) & (b == c)){  
  print("primeira condição")  
} else{  
  print("segunda condição")  
}
```

Listing 41: Exemplos de IF com expressões

segunda condição

Listing 42: Exemplos de IF com expressões



1.6. Estruturas de Repetição

Quando se programa, é comum precisar de um mesmo trecho de código várias vezes. Para evitar repetição desnecessária de código, que muitas vezes pode ser inviável, existem as estruturas de repetição.

1.6.1. For

A estrutura de repetição *for()* permite criar *loops* para casos em que sabemos, de antemão, o número de repetições que devem ser realizadas. Por exemplo, se soubermos previamente que o código deve se repetir 10 vezes, podemos usar o laço *for*. Veja um exemplo no trecho 43:

```
#a sintaxe do for é:
#for( variavel in vetor_numeros ){
# ação
#}

print("primeiro exemplo")
for(i in 1:3){ #estipulando previamente 3 iterações
print(i)
}

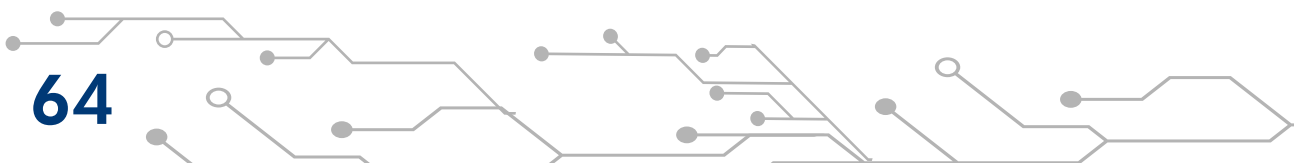
print("segundo exemplo")
for(x in 1:4){ #4 iterações
print("iteração")
}
```

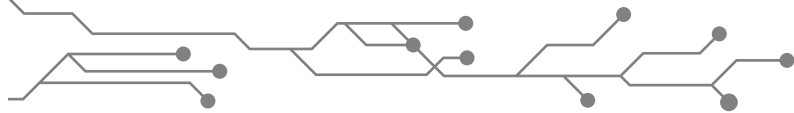
Listing 44: Exemplos de FOR

```
primeiro exemplo
1
2
3
segundo exemplo
iteração
iteração
iteração
iteração
```

Listing 44: Exemplos de FOR

Com o *for()*, também é possível iterar vetores, assunto que será tratado mais à frente.





1.6.2 While

O comando *while()* realiza um teste lógico no início do *loop* e, a cada vez que esse teste retorna ao valor verdadeiro, os comandos associados ao laço são executados. Quando o teste retornar ao valor falso, o laço é interrompido e o *loop* termina. Confira no trecho 45.

```
#sintaxe do while:
#while(condição){
#ação
#}

x <- 3
while(x > 0){
  print(x)
  x <- x - 1
}
```

Listing 45: Exemplos de while

```
3
2
1
```

Listing 46: Exemplos de while

2. AGRADECIMENTOS

O conteúdo deste capítulo foi produzido baseado em Curso-R (2021), Eduardo Ferreira (2021) e Mayer (2021).

3.REFERÊNCIAS

Curso-R, Material -. 2021. R básico.
Eduardo Ferreira, Weverton Lisboa. 2021. Introdução ao R.
Mayer, Fernando. 2021. Entrada e saída de dados no R.



Capítulo 03

Arranjos

1.ARRANJOS

Os arranjos são vários dados em uma única variável.

1.1. Vetores

São uma das estruturas de dados mais importantes da computação. Conceitualmente, os vetores são sequências de dados; ou seja, uma variável que guarda mais de um dado, do mesmo tipo. É possível criar um vetor em R usando a *função c()*, veja o exemplo no trecho 47.

- **Acessando cada elemento**

Como agora é possível guardar vários dados em uma única variável, é interessante saber como acessar individualmente cada um desses dados. Por exemplo, ao criar um vetor com 5 elementos, é possível haver a necessidade de acessar o terceiro elemento e alterar o primeiro. Veja o exemplo no trecho 49:

```
vetor_numerico <- c(1,2,3,4)
vetor_caractere <- c("a","b","c")
```

```
a <- 2
b <- 10
vetor_variaveis <- c(a,b)
print(vetor_numerico)
print(vetor_caractere)
print(vetor_variaveis)
```

Listing 47: Exemplos de vetores

```
1 2 3
"a" "b" "c"
2 10
```

Listing 48: Exemplos de vetores

```
vect <- c(1,10,20,30,40)
print(vect[3]) #acessando o terceiro elemento
vect[1] <- 100 #alterando o primeiro elemento
print(vect)
```

Listing 49: Exemplos de acesso aos elementos do vetor

```
20
100 10 20 30 40
```

Listing 50: Exemplos de acesso aos elementos do vetor

- **Estruturas de repetição com vetores**

Ao começar a trabalhar com vetores, começa a se tornar evidente a necessidade de estruturas de repetição. Por exemplo, é possível acessar todos os elementos de um vetor de tamanho 5 com poucas linhas de código. Segue o exemplo no trecho 51:

```
vect <- c(1,2,3,4,5)
for( i in vect ){
  print(i)
}
```

Listing 51: Exemplos de acesso ao vetor

```
1
2
3
4
5
```

Listing 52: Exemplos de acesso ao vetor

A função *length()* retorna quantos elemento tem um vetor. Com o *while()* e *length()* , pode-se acessar todos os elementos de índice par de um vetor, por exemplo. Confira o exemplo do trecho 53.

```
vect <- c(10,20,30,40,50,60,70,80,90,100)
elemento <- 2

while(elemento <= length(vect)){
  print(vect[elemento])
  elemento <- elemento + 2
}
```

Listing 53: Exemplos de acesso ao vetor

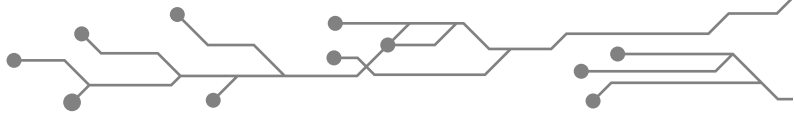
```
20
40
60
80
100
```

Listing 54: Exemplos de acesso ao vetor

No exemplo 54, o vetor tem tamanho 10 e acessamos apenas os elementos nos índices pares. É possível fazer isso graças à variável *elemento*, que vai sendo incrementada de 2 em 2. Perceba que a condição no *while()* permite que a variável *elemento* seja incrementada até 10. É importante delimitar dessa forma para evitar acessos inválidos; ou seja, acessos em posições ainda não criadas no vetor.

1.2 Matrizes

Matrizes podem ser vistas como vetores de duas dimensões. Pode-se pensar como uma tabela, com todas as linhas e colunas do mesmo tipo. Para criar uma matriz, usa-se a função *matrix()* . Veja no trecho 55.



```
#sintaxe da matrix():
#matrix(elementos, numero de linha, numero de colunas)

m <- matrix(1:10, nrow = 2, ncol = 5)
n <- matrix(1:30, nrow = 3, ncol = 10)
print(m)
print(n)
```

Listing 55: Exemplos de matrizes

```
1 2 3 4 5
6 7 8 9 10

 1  2  3  4  5  6  7  8  9 10
11 12 13 14 15 16 17 18 19 20
21 22 23 24 25 26 27 28 29 30
```

Listing 55: Exemplos de matrizes

- **Acessos aos elementos de uma matriz**

Para acessar um elemento específico da matriz, é parecido com acessos de elementos de um vetor. É preciso mostrar explicitamente qual linha e coluna se quer acessar, como é apresentado no trecho 57.

```
m <- matrix(1:10, nrow = 2, ncol = 5)
print(m[1,1])
print(m[2,5])
```

Listing 57: Exemplos de matrizes

```
1
10
```

Listing 58: Exemplos de matrizes

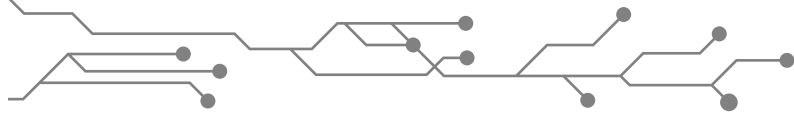
O uso de estruturas de repetição é muito útil para fazer vários acessos.

1.3. Data frames

Semelhante a uma matriz, mas suas colunas e linhas têm nome e podem ter tipos de dados diferentes. É possível criar um *Data Frame* sem ter que importá-lo como um arquivo .csv, com a função `data.frame()`. Veja no trecho 59:

```
1 #sintaxe:
2 #data.frame(elementos_coluna1, elementos_coluna2, ...)
3 funcionarios <- data.frame(nome = c("João", "Maria", "José"),
4   sexo = c("M", "F", "M"),
5   salario = c(1000, 1200, 1300))
6
7 print(funcionarios)
```

Listing 59: Exemplos de DataFrame



	nome	sexo	salario
1	João	M	1000
2	Maria	F	1200
3	José	M	1300

Listing 60: Exemplos de DataFrame

2. AGRADECIMENTOS

O conteúdo deste capítulo foi produzido baseado em Curso-R (2021), Eduardo Ferreira (2021) e Mayer (2021).

3. REFERÊNCIAS

Curso-R, Material -. 2021. R básico.
Eduardo Ferreira, Weverton Lisboa. 2021. Introdução ao R.

Capítulo 04

Funções

1. FUNÇÕES

Funções são ferramentas muito poderosas na programação. São usadas para definir um trecho de código que poderá ser executado posteriormente, quantas vezes for necessário. Sendo assim, uma função só será executada quando for chamada explicitamente pelo programador.

Como R tem uma comunidade bastante ativa, muitas funções com diferentes funcionalidades já foram criadas e estão disponíveis para serem usadas. Portanto, é comum encontrar as funções que você precisa já prontas. É possível pesquisá-las no Google e baixá-las.



Uma dica útil para programar em R é a funcionalidade *help*. Ao digitar *?nome_função* em um *script*, uma janela abrirá com um breve resumo sobre como a função funciona, podendo ser uma forma de esclarecer dúvidas.

Várias funções já lhe foram apresentadas. Uma que pode ser muito útil é a *summary()*, que retorna um breve resumo sobre o dado que recebe como argumento. Veja o exemplo no trecho 61.

Algumas dessas funções prontas não vêm com a implementação exposta; ou seja, você tem a informação necessária para saber qual o parâmetro é necessário e qual a resposta da função, mas não como ela, de fato, foi feita. Mesmo tendo uma vasta coleção de funções já prontas, é importante saber criar as próprias funções.

```
vet1 <- c("a","b","c")
vet2 <- c(1,2,3)
```

```
?summary()
```

```
summary(vet1)
summary(vet2)
```

Listing 61: Exemplos de funções prontas

```
> summary(vet1)
Length Class Mode
3 character character
```

```
> summary(vet2)
Min. 1st Qu. Median Mean 3rd Qu. Max.
1.0    1.5    2.0    2.0    2.5    3.0
```

Listing 62: Exemplos de funções prontas

Para criar uma função, é preciso usar a palavra-chave *function()* . Confira no trecho 63:

```
#sintaxe:
#nome_função <- function(argumento da função){
#  ações
#}

soma <- function(vetor1){                #soma elementos de um vetor
x <- 0
for(i in vetor1){
x <- x + i
}
return(x)    #retorno da funcao

}

vet <- c(1,2,3,4,5)                    #vetor

resultado <- soma(vet) #recebe o resultado da função
print(resultado)
```

Listing 63: Exemplos de funções

15

Listing 64: Exemplos de funções

1.1. Funções Recursivas

Em computação, recursividade é a capacidade de invocar a si mesmo. Nesse aspecto, uma função recursiva consegue chamar-se a si própria para executar uma tarefa.



Um bom exemplo inicial para criar uma função recursiva é uma função que calcula o fatorial de um número. O fatorial de um número é calculado pela multiplicação desse número por todos os seus antecessores, até chegar ao número 1. Veja a implementação no trecho 65.

```
fatorial <- function(n){ #definição da função
if(n == 0 | n == 1) return(1)
return(fatorial(n-1)*n) #perceba que a função chama ela mesma
#mas com um argumento diferente
}

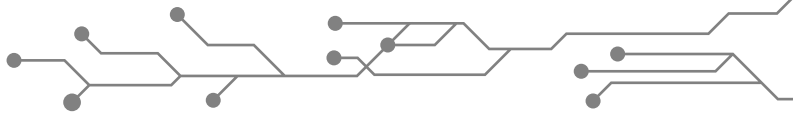
print(fatorial(5))
```

Listing 66: Exemplos de recursividade

120

Listing 66: Exemplos de recursividade

Em geral, funções recursivas contam com menos código, mas, muitas vezes, são mais difíceis de entender.



O primeiro passo para criar uma função recursiva é detectar o *caso base*; ou seja, aquele que será o sinal de que a função deve acabar. No exemplo anterior, o *caso base* era o número 0 e o número 1. Isso acontece para evitar um *loop*. Se o *caso base* não tivesse sido especificado, o programa entraria em *loop*, pois continuaria decrementando e multiplicando os números infinitamente, e é sabido que não existe fatorial de números negativos.

2. AGRADECIMENTOS

O conteúdo deste capítulo foi produzido baseado em Curso-R (2021), Eduardo Ferreira (2021) e Mayer (2021).

3. REFERÊNCIAS

Curso-R, Material -. 2021. R básico.
Eduardo Ferreira, Weverton Lisboa. 2021. Introdução ao R.



06

Exercícios

1. ELEMENTOS BÁSICOS DA LINGUAGEM PYTHON

1) A fórmula para calcular a área de uma circunferência é: $\text{área} = \pi \cdot \text{raio}^2$. Considerando para este problema que π é constante e vale 3.14159:

2) Leia 2 valores numéricos A e B, que correspondem a 2 notas de um aluno. A seguir, calcule a média do aluno, sabendo que a nota A tem peso 3.5 e a nota B tem peso 7.5 (A soma dos pesos, portanto é 11). Assuma que cada nota pode ir de 0 até 10.0, sempre com uma casa decimal.

3) Leia 3 valores, no caso, variáveis A, B e C, que são as três notas de um aluno. A seguir, calcule a média do aluno, sabendo que a nota A tem peso 2, a nota B tem peso 3 e a nota C tem peso 5. Considere que cada nota pode ir de 0 até 10.0, sempre com uma casa decimal.

4) Escreva um programa que leia o número de um funcionário, seu número de horas trabalhadas, o valor que recebe por hora e calcule o salário desse funcionário. A seguir, mostre o número e o salário do funcionário, com duas casas decimais.

5) Leia quatro valores inteiros A, B, C e D. A seguir, calcule e mostre a diferença do produto de A e B pelo produto de C e D segundo a fórmula: $\text{DIFERENÇA} = (A * B - C * D)$.

2. COMANDOS BÁSICOS DA LINGUAGEM PYTHON

1) Faça um programa que imprima na tela os números de 1 a 20, um abaixo do outro.

2) Faça um Programa que leia um vetor de 10 números reais e mostre-os na ordem inversa.

3) Faça um programa que solicite a data de nascimento (dd/mm/aaaa) do usuário e imprima a data com o nome do mês por extenso.

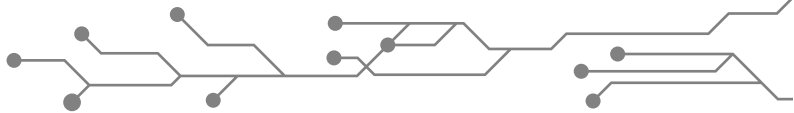
4) Faça um programa que leia a idade de 5 pessoas e imprima na tela quantas já atingiram a maioridade.

5) Escreva um programa que receba 2 valores inteiros referentes à largura e altura de um retângulo e imprima na tela um retângulo com essas dimensões.

3. ARRANJOS

1) Ordenar uma lista de valores pode ser muito útil e tem diversas aplicações na computação. Existem diversos algoritmos de ordenação: alguns mais simples, outros mais complexos, que envolvem conceitos não apresentados neste curso. O *bubble sort* é um algoritmo de ordenação dos mais simples. A ideia é percorrer a lista diversas vezes e, a cada passagem, fazer flutuar para o topo o maior elemento da sequência. Dada uma lista de valores em ordem aleatória, use o algoritmo *bubble sort* para ordená-la e imprima o resultado.





2) Faça um programa que leia um vetor $X[10]$. Substitua, a seguir, todos os valores nulos e negativos do vetor X por 1. Em seguida, mostre o vetor X .

3) Faça um programa que leia 2 vetores de tamanho 5 e crie um terceiro que tenha todos os elementos desses vetores.

4) Faça um programa que leia um valor T e preencha uma lista $N[20]$ com a sequência de valores de 0 até $T-1$ repetidas vezes, conforme exemplo: $T3 = 3$, vetor = [0, 1, 2, 0, 1, 2, 0, 1, 2, 0, 1, 2, 0, 1, 2, 0, 1, 2, 0, 1]

5. Nome na vertical. Faça um programa que solicite o nome do usuário e imprima-o na vertical.

4. FUNÇÕES

1) Faça um programa, com uma função que receba um argumento e retorne 'P', se o argumento for positivo, ou 'N', se o argumento for negativo.

2) Faça uma função que retorne ao fatorial de um número N maior que 0. Agora refaça o problema proposto de modo recursivo.

3) Faça uma função que retorne à soma dos N primeiros números da sequência de Fibonacci. Agora, refaça o problema proposto de maneira recursiva.

4) Faça uma função que receba um vetor de números inteiros e retorne a soma dos elementos do vetor.

5) Faça uma função que receba um vetor de números inteiros e retorne quantos desses números são pares.

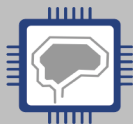
6. AGRADECIMENTOS

O conteúdo deste capítulo foi produzido baseado em Curso-R (2021), Eduardo Ferreira (2021) e Mayer (2021).

7. REFERÊNCIAS

Curso-R, Material -. 2021. R básico.

Eduardo Ferreira, Weverton Lisboa. 2021. Introdução ao R.



Octave

**Editado por
Amanda Rossinol
Sabrina de A. Silveira**



Capítulo 01

Elementos básicos da linguagem Octave

1. VARIÁVEIS

Variáveis em Octave, assim como em outras linguagens, permitem que valores sejam armazenados e referenciados dentro de um programa.

```
1 octave:1>first_number = 3;  
2 octave:2>second_number = 25;  
3 octave:3> variable1 = 37;
```

Listing 1: Declaração de variáveis em Octave.

Podemos criar variáveis usando um comando de atribuição, como mostrado em 1. Nele, o nome da variável está à esquerda do símbolo =, e o valor a ser armazenado à direita. Importante notar que não são especificados tipos às variáveis no momento da atribuição. Portanto, em Octave, um número é interpretado como real, a menos que seja explicitado o contrário.

A nomeação de variáveis deve seguir algumas convenções:

1. Deve iniciar com uma letra do alfabeto;
2. Pode conter letras, dígitos e *underscores*;
3. Não pode conter espaços;
4. É *case sensitive*, ou seja, há diferenciação entre letras maiúsculas e minúsculas;
5. Palavras reservadas da linguagem não podem ser usadas como variáveis.

2. TIPOS DE DADOS

Os tipos de dados padrão da linguagem são: escalares e matrizes complexas/reais, intervalos, *strings* de caracteres e tipos estruturados de dados. Nesta seção, são apresentados apenas os tipos numéricos. *Strings* e *Matrizes* serão discutidos no Capítulo 3.

2.1. Números

2.1.1. Escalares

Constantes numéricas mais simples, podem ter um único número inteiro, fração decimal, notação científica ou números complexos.


```

1 octave:1> typeinfo(235)
2 ans = scalar
3 octave:2> typeinfo(23.5)
4 ans = scalar
5 octave:3> typeinfo(2.35e+2)
6 ans = scalar
7 octave:4> typeinfo(2 + 5i)
8 ans = complex scalar
9 octave:5> typeinfo(2 + 5 i)
10 parse error:
11     syntax error
12 >>> typeinfo(2 + 5 i)
13                ^

```

Listing 2: Tipos de dados numéricos - acesso em 26 de janeiro de 2021, adaptado de http://laris.fesb.hr/digitalno_vodjenje/octave/-doc/octave_5.html SEC50

Em programa 2, é utilizada a função *typeinfo()* para apresentar o tipo dos valores inseridos. Funções serão discutidas com detalhes no capítulo 3.



Note que na representação de escalares complexos não pode haver espaços entre o número e a constante imaginária *i*, sendo apontado um erro de sintaxe.

2.1.2. Intervalos

Um intervalo é uma forma conveniente de escrever um vetor com elementos igualmente espaçados. É definido pelo valor do primeiro elemento no intervalo, um valor incremental opcional e um valor máximo que os elementos no intervalo não devem ultrapassar.

A base, o incremento e o limite são separados pelo caracter `:` e podem conter expressões e chamadas de funções. Se o incremento não for especificado, tem valor igual a 1.

```

1 octave:1> 1:5
2 ans =
3     1     2     3     4     5
4 octave:2> rangeVar = 1:2:20
5 rangeVar =
6     1     3     5     7     9    11    13    15    17    19

```

Listing 3: Exemplo de intervalos

O exemplo 3 mostra que intervalos podem ser armazenados em variáveis, como é feito na atribuição a *rangeVar*.

2.2 Constantes

Octave tem a capacidade de armazenar constantes, que são valores previamente conhecidos e não podem ser alterados. Alguns exemplos de constantes são listados em 4.

```
1 octave:1> pi
2 ans = 3.1416
3 octave:2> eps
4 ans = 2.2204e-16
5 octave:3> inf
6 ans = Inf
7 octave:4> i
8 ans = 0 + 1i
9 octave:5> j
10 ans = 0 + 1i
11 octave:6> realmin
12 ans = 2.2251e-308
13 octave:7> realmax
14 ans = 1.7977e+308
```

Listing 4: Constantes Octave

Os valores *realmax* e *realmin* indicam, respectivamente, o maior e menor número real positivo utilizável.

3. EXPRESSÕES ARITMÉTICAS E LÓGICAS

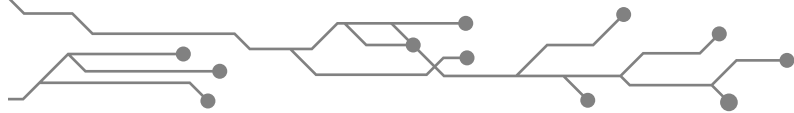
Expressões podem ser criadas usando valores, variáveis já criadas, operadores, funções e parênteses.

3.1 Operadores aritméticos

Podemos utilizar os operadores da Tabela 1 em Octave para lidar com os valores numéricos.

Tabela 1: Operadores aritméticos

Operador	Descrição	Exemplo
+	Adição	$x + y$
-	Subtração/Negação	$x - y$
*	Multiplicação	$x * y$
/	Divisão	x / y
^	Potenciação (x elevado a y)	$x ^ y$



Alguns exemplos são listados em 5.

```
1 octave:1> 2*3
2 ans = 6
3 octave:2> 5-2
4 ans = 3
5 octave:4> 2^2
6 ans = 4
7 octave:4> 2^2
8 ans = 4
9 octave:5> a=7; b=2;
10 octave:6> a+b
11 ans = 9
```

Listing 5: Exemplo de operações aritméticas

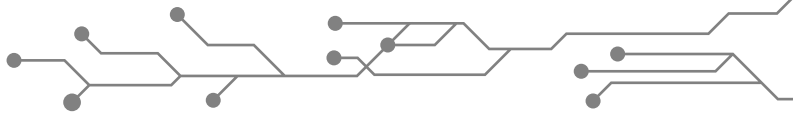
3.2 Operadores relacionais

A Tabela 2 mostra os operadores relacionais da linguagem que retornam um tipo lógico após a avaliação da expressão.

Tabela 2: Operadores relacionais

Operador	Descrição	Exemplo
>	Maior que: True se o operando à esquerda é maior que o à direita	$x > y$
>=	Maior ou igual: True se operando à esquerda é maior ou igual ao à direita	$x \geq y$
<	Menor que: True se o operando à esquerda é menor que o à direita	$x < y$
<=	Menor ou igual: True se operando à esquerda é menor ou igual ao operando à direita	$x \leq y$
==	Igualdade: True se ambos operandos são iguais	$x == y$
!=	Diferente: True se os operandos são diferentes	$x != y$

O valor retornado é 1, se a comparação for verdadeira, e 0, caso contrário.



Alguns exemplos de uso dos operadores podem ser vistos em 6.

```
1 octave:1> a = 11; b = 13; c = 2; d = 9;
2 octave:2> a == b
3 ans = 0
4 octave:3> c != d
5 ans = 1
6 octave:4> a+b == c
7 ans = 0
8 octave:5> a+c == b
9 ans = 1
10 octave:6> 9 < 1
11 ans = 0
12 octave:7> 5 <= 5
13 ans = 1
```

Listing 6: Exemplo de operações aritméticas

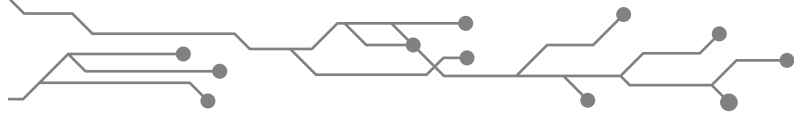
3.3 Precedência de operadores

A precedência de operadores determina como os operadores são agrupados quando mais de um deles aparecem próximos em uma expressão.

Tabela 3: Precedência de operadores

Operação
Parênteses
Exponenciação
Multiplicação e Divisão
Adição e Subtração
Operadores Relacionais
Atribuição

No exemplo 7, podemos ver que, no comando 1, primeiramente, é executada a multiplicação $5*2$ e depois a adição e subtração. Assim, temos $7 + 10 - 3$. Já no comando 2, devido ao uso de parênteses, a adição é a primeira a ser calculada, restando assim $12 * 2 - 3$.



```
1 octave:1> 7+5*2-3
2 ans = 14
3 octave:2> (7+5)*2-3
4 ans = 21
```

Listing 7: Precedência de operadores

4. COMENTÁRIOS

Comentários são utilizados para explicar trechos de código. Para inserir comentários nos códigos, devemos iniciar a linha com % ou # para que ela seja ignorada pelo interpretador, como mostra o exemplo 8.

```
1 octave:1> var_1 = 'Existem'; num = ' 3'; #declaração de strings
2 octave:2> var_2 = ' flores no jardim.'; #declaração de stings
3 octave:43> [var_1, num, var_2] %concatenação de strings
4 ans = Existem 3 flores no jardim.
```

Listing 8: Exemplo de comentários

5.AGRADECIMENTOS

O conteúdo deste capítulo foi produzido baseado em Hansen (2019), Octave (2020) e GeeksforGeeks (2021).

6.REFERÊNCIAS

GeeksforGeeks. 2021. *How to take input in Octave GNU?*
Hansen, Jesper. 2019. *GNU Octave Beginner's Guide*. PACKET Publishing.
Octave, GNU. 2020. *Aprender GNU Octave*.



Capítulo 02

Comandos básicos da linguagem Octave

1. COMANDOS DE ATRIBUIÇÃO

Expressões de atribuição são usadas para armazenar valores em variáveis. O operador mais comum é o `=`, podendo ser combinado com outros operadores matemáticos para encurtar expressões, tais como `+=`, `*=`, `-=` e `/=`. Alguns exemplos de comandos de atribuição são mostrados em 9.

```
1 >> var = 5
2 var = 5
3 >> var += 4 # como se fosse var = var + 4
4 var = 9
5 >> var -= 6 # como se fosse var = var - 6
6 var = 3
7 >> var *= 2 # como se fosse var = var * 2
```

Listing 9: Exemplo de intervalos

2. ENTRADA E SAÍDA PADRÃO

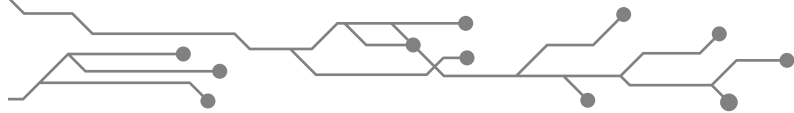
2.1. Entrada

Octave tem funções que facilitam a entrada de dados do usuário pelo *prompt*. A mais simples entre elas é a função *input*, que apresenta a mensagem desejada e aguarda até que algo seja digitado e a tecla *Enter* seja pressionada. Assim, o valor é atribuído à variável especificada, como mostra o exemplo 10.

```
1 octave:1> value = input('Insira um valor:')
2 Insira um valor:12
3 value = 12
```

Listing 10: Entrada simples de dados

Por padrão, a função *input* lê os valores inseridos como números, vetores ou matrizes. Portanto, caso a intenção seja ler uma *string*, o parâmetro `'s'` deve ser especificado, como em 11.



```
1 octave:1> value = input('Insira o nome do seu pet:', 's')
2 Insira o nome do seu pet:Bob
3 value = Bob
```

Listing 11: Entrada de strings utilizando *input*

É possível ainda utilizar as funções *menu*, que recebe como parâmetro um título e as opções separadas por vírgula e *yes_or_no*, que recebe o título e apresenta *sim* e *não* como opções de escolha. Ambas são exemplificadas em 12.

```
1 Deseja prosseguir?(yes or no) no
2 ans = 0      % opção no retorna o valor 0, ou false
3 octave:2> yes_or_no('Deseja prosseguir?')
4 Deseja prosseguir?(yes or no) yes
5 ans = 1      % opção yes retorna 1, ou true
```

Listing 12: Exemplo utilizando *menu* e *yes_or_no*

2.2 Saída

Normalmente, o valor de uma expressão é impresso em tela assim que é avaliado, como mostra 13:

```
1 octave:1> name = 'John'
2 name = John
3 octave:2> name
4 name = John
```

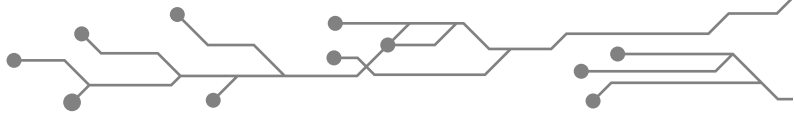
Listing 13: Saída simples de dados

Essa forma funciona bem, porém, juntamente com o valor da variável também é exibido seu nome (ou a variável *ans*). Para impressão apenas do valor, pode ser utilizada a função *disp*. Seu uso é exemplificado em 14.

```
1 octave:1> disp(name)
2 John
```

Listing 14: Função *disp*

É possível formatar a saída utilizando o comando *printf*. Este comando afeta apenas a exibição dos números, mas seu valor armazenado permanece inalterado.



A instrução *printf* recebe um *template*, e os valores a apresentar, como é mostrado em 15. Note que mais de um *template* pode ser usado na função, como é o caso do *%d* mostrado, que representa um valor inteiro.

```
1 octave:1> largura = 2;
2 octave:2> comprimento = 5;
3 octave:3> printf("0 retângulo tem %d de largura e %d
4               de comprimento.\n", largura, comprimento);
5 0 retângulo tem 2 de largura e 5 de comprimento.
```

Listing 15: Saída formatada com a função *disp* - acesso em 27 de Janeiro de 2020, adaptado de http://octave.di.uminho.pt/index.php/Ler_e_apresentar_dados

3. ENTRADA E SAÍDA A PARTIR DE ARQUIVOS

3.1. Abertura

O comando *fopen* é utilizado para abrir um arquivo e retornar o seu ID. Normalmente, dois parâmetros de entrada são necessários: *string* com nome do arquivo e identificador de operação (*r* para leitura, *w* para escrita, *a* para acrescentar e *rb* para leitura de binários). O exemplo 16 mostra como é feita a abertura de um arquivo em modo de escrita.

```
1 octave:1> file = fopen('arquivoTexto.txt', 'w')
2 file = 25
```

Listing 16: Abertura de arquivo em modo de escrita

3.2 Fechamento

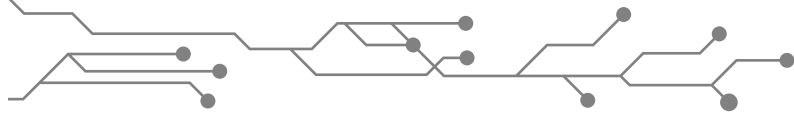
É sempre importante fechar os arquivos abertos no programa assim que não sejam mais necessários. Para fechar um arquivo, basta utilizar o comando *fclose*, como em 17.

```
1 octave:1> fclose(file);
```

Listing 17: Fechamento de arquivo

3.3 Escrita

Para escrever em arquivos da forma que possa ser aberto e entendido (não binária), é usada a função *fprintf*. Com ela, o formato de *string* é necessário e, por isso, pode ser mais lenta. Um exemplo de uso está em 18, onde o arquivo *texto.txt* é aberto em modo de escrita (*w*) e recebe o conteúdo da variável *content*. A função *type* mostra o que está escrito no arquivo.



```
1 octave:1> content = "Minha casa é a de número 140.";
2 file = 25
3 octave:2> fprintf(file, "%s", content)
4 octave:30> fclose(file)
5 ans = 0
6 octave:3> type('texto.txt')
7 Minha casa é a de número 140.
```

Listing 18: Escrita em arquivo utilizando *fprintf*

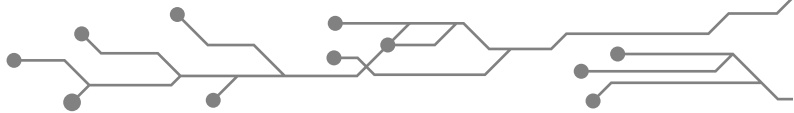
3.4 Leitura

Um método de leitura em arquivo oferecido pelo Octave é por meio da função *fread*. Utilizando o arquivo *texto.txt*, criado nos exemplos anteriores, vemos um exemplo de uso do comando em 19.

```
1 octave:1> file = fopen('texto.txt', 'r')
2 file = 21
3 octave:2> fread(file)
4 ans =
5     77
6    105
7    110
8     32
9     ...
10    49
11    52
12    48
13    46
```

Listing 19: Leitura de arquivo utilizando *fread*

Como o arquivo foi escrito utilizando *fprintf* a saída do comando *fread* é dada em valores ASCII. Importante destacar que agora é utilizado *r* para abrir o arquivo em modo de leitura. Uma alternativa a esse comando é o comando *fgets*, mostrado em 20, que lê linha por linha do arquivo. Uma vez alcançado o fim do arquivo, a função retorna ao valor -1.



```
1 octave:1> file = fopen('texto.txt', 'r')
2 file = 19
3 octave:2> fgets(file)
4 ans = Minha casa é a de número 140.
5 octave:3> fgets(file)
6 ans = -1
```

Listing 20: Leitura de arquivo utilizando *fgets*

4. ESTRUTURAS CONDICIONAIS

Estruturas condicionais são estruturas de controle que verificam se determinada expressão é verdadeira ou não. Caso seja verdadeira, os comandos seguem sendo executados. A estrutura condicional da linguagem é o *if*, assim como em outras linguagens, que é apresentado em 21.

```
1 val_1 = 12; val_2 = 14;
2 octave:14> if(val_1 != val_2),
3             > val_2 = val_2 -2,
4             > endif
5 val_2 = 12
```

Listing 21: Comando condicional *if*

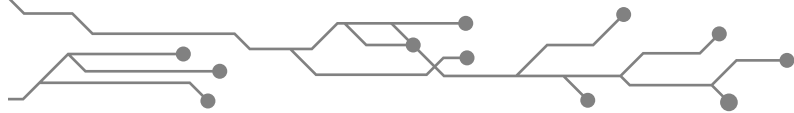
Algumas variações também podem ser utilizadas, como o *if-else*, que executa os comandos em *else*, caso a verificação feita em *if* falhe. Um exemplo de uso pode ser visto em 22.

```
1 octave:1> val = 21;
2 octave:2> if val > 20, disp('Valor maior que 20');
3             > else disp('Menor ou igual a 20');
4             > endif
5 Valor maior que 20
```

Listing 22: Comando condicional *if-else*

5. ESTRUTURAS DE REPETIÇÃO

Um problema que pode surgir durante o uso de comandos de repetição são **os ciclos infinitos**, que são aqueles que executam indefinidamente, pois não atingem determinada condição de parada. Nesses casos, é possível encerrar a execução com o uso de *Ctrl+C*.



5.1 While

O *while* executa repetidamente enquanto a condição que o segue for verdadeira. Nele, primeiro, verifica-se a condição e, depois, os comandos são executados. Sendo assim, o ciclo pode nem ser executado, se a condição for falsa já no começo. Um exemplo da utilização de *while* é apresentado no código 23:

```
1 octave:1> num = input("Insira um número: ");
2   Insira um número: 16
3 octave:2> while (num > 1)
4     > num = num/2
5     > endwhile
6 num = 8
7 num = 4
8 num = 2
9 num = 1
```

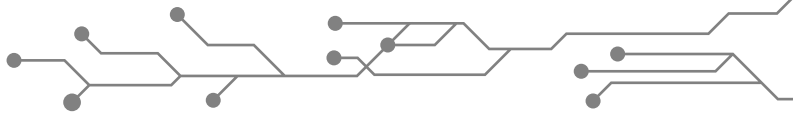
Listing 23: Estrutura de repetição *while*. O exemplo mostra o número de divisões por 2 feitas para o valor de entrada. Acesso em janeiro de 2021, adaptado de - http://octave.di.uminho.pt/index.php/Os_ciclos_while

5.2 For

Os comandos do *for* são executados enquanto a condição que o compõe for verdadeira. Em 24, podemos ver que, na primeira vez que a condição é testada, o valor de *i* é igual a 1. Na segunda iteração, como o incremento é igual a 2, *i* é igual a 3, e assim por diante. Na sexta iteração, *i* assume valor 11, que é maior que 10 e, portanto, os comandos não são mais executados.

```
1 octave:1> for i = 1:2:10, # intervalo com incremento de 2
2     > disp(i),
3     > end;
4   1
5   3
6   5
7   7
8   9
```

Listing 24: Estrutura de repetição *for*



Também é possível definir o *for* com decremento dos valores, como mostra 25:

```
1 octave:3> for i = 10:-2:1,
2   > disp(i),
3   > end;
4   10
5   8
6   6
7   4
8   2
```

Listing 25: Estrutura de repetição *for* com decremento

6.AGRADECIMENTOS

O conteúdo deste capítulo foi produzido baseado em Hansen (2019), Octave (2020) e Kumar (2019).

7.REFERÊNCIAS

Kumar, Purnendu. 2019. *GNU Octave*.

Hansen, Jesper. 2019. *GNU Octave Beginner's Guide*. PACKET Publishing.

Octave, GNU. 2020. *Aprender GNU Octave*.



Capítulo 03

Arranjos

1. STRINGS

Strings são sequências de caracteres que podem ser definidos usando tanto *aspas simples* (apóstrofo) quanto *aspas duplas*. Um exemplo é apresentado no código 26.

```
1 octave:1> 'text'
2 ans = text
3 octave:2> "text"
4 ans = text
```

Listing 26: Strings definidas com aspas duplas ou simples.

As *strings* podem ser de qualquer tamanho; não há limitações. Como podem conter caracteres especiais, ao utilizar *strings* com aspas duplas, é necessário o uso de \ para inseri-los, como em 27.

```
1 octave:4> s = "Esta string é formada\nde duas linhas"
2 s = Esta string é formada
3 de duas linhas
```

Listing 27: Inserção de caracteres especiais em strings.

1.1 Concatenação de strings

Há duas formas de concatenar *strings*, usando `[]` ou `strcat()`, conforme mostrado no código 28.

```
1 octave:1> str_3 = ['String 1 ' 'String 2']
2 str_3 = String 1 String 2
3
4 octave:1> str_4 = strcat('String 1 ', 'String 2')
5 str_4 = String 1 String 2
```

Listing 28: Concatenação de strings.



1.2. Comparação de strings

A função `strcmp()` é usada para comparar duas *strings*. Há mais de uma variação na função, como em 29.

```
1 octave:1> str_1 = 'banana'; str_2 = 'batata'; str_3='banana';
2 octave:2> strcmp(str_1, str_2)
3 ans = 0
4 octave:3> strcmp(str_1, str_3)
5 ans = 1
```

Listing 29: Comparação de strings.

2. MATRIZES E VETORES

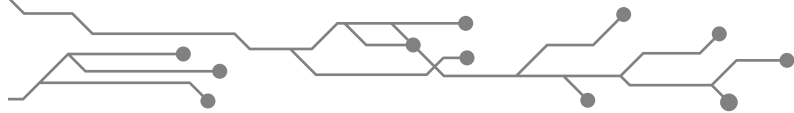
A definição de matrizes/vetores é bastante simples, pois suas dimensões são determinadas automaticamente, com base nos dados. Os elementos devem ser digitados entre colchetes `[]`. Assim, elementos de mesma linha são separados por vírgula ou espaço, e as linhas são separadas por ponto e vírgula. Apresentamos exemplos de criação de matrizes no código 30.

```
1 octave:1> mat_1 = [ 1, 2, 3; 4, 5 , 6 ]
2 mat_1 =
3      1  2  3
4      4  5  6
5 octave:2> vec = [7 8 9]
6 vec =
7      7  8  9
```

Listing 30: Declaração de matriz

2.1. Comandos de criação especiais

É possível a criação de matrizes especiais, como matriz identidade, matriz preenchida com 0's, 1's ou valores aleatórios, como mostrado nos trechos de código 31, 32, 33 e 34.



```
1 octave:1> mat_1 = ones(2)
2 mat_1 =
3
4     1     1
5     1     1
```

Listing 31: Matriz preenchida com 1

```
1 octave:1> mat_0 = zeros(2)
2 mat_0 =
3
4     0     0
5     0     0
```

Listing 32: Matriz preenchidas com 0

```
1 octave:1> mat_I = eye(2)
2 mat_I =
3
4 Diagonal Matrix
5
6     1     0
7     0     1
```

Listing 33: Matriz Identidade

```
1 octave:1> mat_rand = rand(2)
2 mat_rand =
3
4     0.073424    0.499395
5     0.500693    0.255697
```

Listing 34: Matriz inicializada com valores aleatórios

2.2 Indexação

Para ter acesso a valores em posições específicas de um vetor/matriz, utilizamos o nome da variável seguida de parênteses, que contêm a posição desejada, como apresenta o código 35.

```
1 octave:1> M = [1 2 3;4 5 6]
2 M =
3     1     2     3
4     4     5     6
5 octave:2> V = [7, 8, 9, 10]
6 V =
7     7     8     9    10
8 octave:3> V(1)
9 ans = 7
10 octave:4> V([1,3]) #valores nas posições 1 e 3
11 ans =
12     7     9
13 octave:5> M([2], [3]) #valor na segunda linha e terceira coluna
14 ans = 6
```

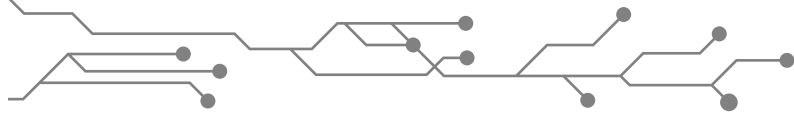
Listing 35: Acesso a posições de vetor/matriz

2.3 Operações com matrizes

Assim como há operadores para escalares, podemos operar com matrizes, de acordo com a tabela 1.

Tabela 1: Operações com matrizes

Operador	Descrição	Exemplo
+	Adição	$X + Y$
-	Subtração	$X - Y$
*	Multiplicação	$X * Y$ ou $k * X$
^	Potenciação	X^k
'	Transposição	X'
	Coluna j da matriz A	$X(:,j)$
	Linha i da matriz A	$X(i,:)$
	Elemento na posição i,j	$X(i,j)$



Alguns exemplos de operações são mostrados em 36.

```
1 octave:1> M = [1 2 3;4 5 6]
2 M =
3     1     2     3
4     4     5     6
5 octave:2> M'
6 ans =
7     1     4
8     2     5
9     3     6
10 octave:3> 2*M
11 ans =
12     2     4     6
13     8    10    12
14 octave:4> M+M+M
15 ans =
16     3     6     9
17    12    15    18
```

Listing 36: Operações com matrizes

3.AGRADECIMENTOS

O conteúdo deste capítulo foi produzido baseado em Hansen (2019), Octave (2020), GeeksforGeeks (2021) e Kumar (2019).

4.REFERÊNCIAS

GeeksforGeeks. 2021. *How to take input in Octave GNU?*
Hansen, Jesper. 2019. *GNU Octave Beginner's Guide*. PACKET Publishing.
Kumar, Purnendu. 2019. *GNU Octave*.
Octave, GNU. 2020. *Aprender GNU Octave*.



Capítulo 04

Funções

1. DEFINIÇÃO DE FUNÇÕES

Instruções sucessivas formam um programa. Se o número de instruções é muito grande, a legibilidade do programa pode ser comprometida. Uma forma de melhor organizar os programas é por meio da criação de funções.

As funções desviam o fluxo do programa para o trecho de código definido, realizam a tarefa que lhe é designada e depois retornam à execução para o trecho de código que a chamou.

Em Octave, funções são definidas seguindo a sintaxe especificada em 4.1, onde *results* é uma lista de valores que serão retornados pela função, ao fim de sua execução. *function_name* é o nome que deve ser usado para chamar a função e os *args* são uma lista de valores passados à função para que seja executada.

$function[results] = function_name(args)$
--

(4.1)

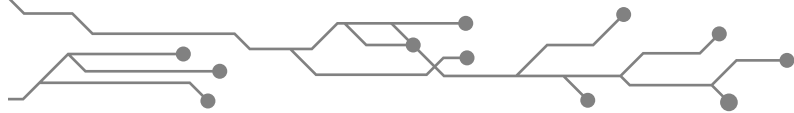
Em 37, vemos o exemplo de uma função simples que ao ser chamada imprime uma mensagem, sem receber ou retornar valores.

```
1  #Definição da função
2  octave:1> function imprime_hello()
3  > disp('Hello!')
4  > endfunction
5
6  #Chamada à função
7  octave:2> imprime_hello()
8  Hello!
```

Listing 37: Exemplo de função sem passagem de parâmetros e retorno de valores.

2. DEFINIÇÕES LOCAIS E GLOBAIS

Por definição, as variáveis em um programa são globais, ou seja, válidas dentro e fora de funções. Porém, quando declaradas dentro das funções, elas passam a ter escopo local. Então, ainda que exista uma variável de mesmo nome fora da função, as alterações feitas não irão afetá-la.



A situação é ilustrada no exemplo 38.

```
1 octave:1> function change_val()
2 > val = 5;
3 > endfunction
4
5 octave:2> val = 3
6 val = 3
7 octave:3> change_val() # a chamada deveria alterar o valor de val para 5
8                        # se o escopo da variável fosse global
9 octave:9> disp(val)
10 3
```

Listing 38: Exemplo de definição global e local.

3. PASSAGEM DE PARÂMETROS

Geralmente, é interessante especificar parâmetros para as funções. Esses parâmetros devem ser especificados dentro dos parênteses que vêm logo após o nome da função em sua definição. Em 39, vemos dois exemplos de funções que recebem parâmetros:

```
1 # função que recebe um valor e imprime seu dobro
2 octave:1> function dobro(x)
3 > disp(2*x)
4 > endfunction
5 octave:2> dobro(3)
6 6
7
8 #função que recebe altura e largura e imprime área
9 octave:3> function area (altura, largura)
10 > disp(altura*largura)
11 > endfunction
12 octave:4> area(2,4)
13 8
```

Listing 39: Exemplo de funções que recebem parâmetros.

4. RETORNO DE VALORES

As funções podem receber um valor ou um vetor de valores. Para isso, os valores a serem retornados devem ser especificados durante a declaração da função, como mostrado em 40. Nele, a função recebe como parâmetro um vetor e retorna a soma de seus elementos. Note que os valores assumidos pelas variáveis dentro da função são os valores a serem retornados.

```

1 octave:1> function soma = calcula_soma(vetor)
2   > soma = sum(vetor);
3   > endfunction
4 octave:2> s = calcula_soma([1,2,3,4])
5 s = 10

```

Listing 40: Exemplo de funções com retorno de valores.

5. FUNÇÕES ELEMENTARES

A linguagem tem diversas funções, em sua maioria matemáticas, já implementadas para uso. Alguns exemplos estão na Tabela 1.

Tabela 1: Funções elementares

Função	Descrição
abs(x)	Módulo de x
acos(x)	Arco cosseno de x
cos(x)	Cosseno de x
sin(x)	Seno de x
round(x)	Arredonda o valor de x
tan(x)	Tangente de x
exp(n)	Função exponencial
log10(x)	Logaritmo de x na base 10
sum(x)	Somatória de x
prod(x)	Produtória de x
sumsq(x)	Somatória dos quadrados dos elementos de x

O exemplo 41 mostra o uso de algumas dessas funções.

```

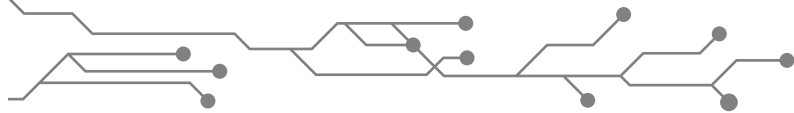
1 octave:1> abs(-5)
2 ans = 5
3 octave:2> log10(100)
4 ans = 2
5 octave:3> exp(1)
6 ans = 2.7183

```

Listing 41: Exemplo de funções matemáticas já definidas na linguagem.

6. AGRADECIMENTOS

O conteúdo deste capítulo foi produzido baseado em Hansen (2019), Octave (2020) e Kumar (2019).

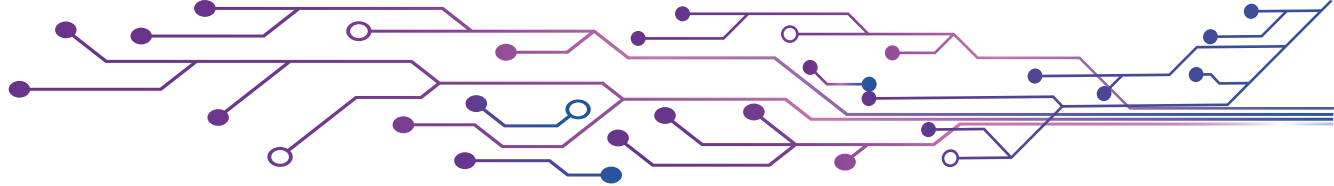


7.REFERÊNCIAS

Hansen, Jesper. 2019. *GNU Octave Beginner's Guide*. PACKET Publishing.

Kumar, Purnendu. 2019. *GNU Octave*.

Octave, GNU. 2020. *Aprender GNU Octave*.



05

Exercícios

1. ELEMENTOS BÁSICOS DA LINGUAGEM OCTAVE

1) Escreva um programa que tenha três variáveis diferentes, sendo que cada uma deve armazenar um valor escalar, escalar complexo ou um intervalo. Use a função `typeinfo` para mostrar o tipo de cada uma delas.

2) Escreva um programa que calcule o fatorial de 5.

3) Crie uma variável intervalo que armazene os valores de 10 a 100, com incremento de 2.

Dica: utilize elementos básicos da linguagem.

4) Escreva um programa que diz se a variável x é maior que y . Os valores de x e y podem ser arbitrários.

5) Usando a constante π , calcule a área de um círculo de raio 5.

2. COMANDOS BÁSICOS DA LINGUAGEM OCTAVE

1) Use estruturas condicionais e de repetição para calcular o máximo divisor comum entre dois valores de entrada. O algoritmo consiste em subtrair do maior o menor, até que ambos sejam iguais.

2) Faça um programa que receba como entrada o nome e idade de uma pessoa, armazene-os e imprima-os na tela.

3) Escreva um programa que imprime o dobro dos valores de 1 até 10, utilizando a estrutura de repetição `while`.

Dica: utilize uma variável como contador e incremente-a em cada iteração do laço.

4) Escreva um programa que, dados a e b , diz se $a > b$, $a < b$ ou $a = b$.

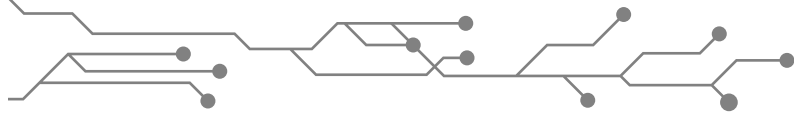
Dica: utilize estruturas condicionais.

5) Abra um arquivo em modo de escrita e escreva nele uma sequência de números. Depois, leia do mesmo arquivo os valores escritos. Lembre-se de sempre fechar os arquivos quando não forem mais necessários.

3. ARRANJOS

1) Crie duas matrizes com 4 linhas e 3 colunas, preenchidas com valores inteiros de sua escolha. Depois some-as e multiplique o resultado por 2.

2) Faça um programa que leia um nome de usuário e a sua senha e não aceite a senha igual ao nome do usuário.



3) Crie uma matriz 3x3 inicializada com valores aleatórios e, então, calcule e exiba sua transposta.

4) Crie um vetor de 10 colunas com valores de sua preferência. Selecione apenas os valores das posições 3,4,7 e 9 e armazene em uma nova variável.

5) Crie uma matriz 4x4 e imprima os elementos da sua diagonal principal.

4. FUNÇÕES

1) Faça uma função que imprima de 1 a 10, cada valor em uma linha.

2) Defina uma nova função chamada *areadocirculo* que calcula a área de um círculo. O único argumento é o raio do círculo.

3) Defina uma função que, dados o número de horas e de minutos, retorne ao número de segundos equivalentes.

4) Faça uma função que receba um vetor de números inteiros e retorne a soma dos elementos do vetor.

5) Usando as funções elementares, calcule seno, cosseno e tangente dos ângulos 30°, 60° e 90°.

5. AGRADECIMENTOS

O conteúdo deste capítulo foi produzido baseado em Hansen (2019), Octave (2020), GeeksforGeeks (2021) e Kumar (2019).

6. REFERÊNCIAS

GeeksforGeeks. 2021. *How to take input in Octave GNU?*
Hansen, Jesper. 2019. *GNU Octave Beginner's Guide*. PACKET Publishing.
Kumar, Purnendu. 2019. *GNU Octave*.
Octave, GNU. 2020. *Aprender GNU Octave*.



Coordenadoria de
Educação Aberta e a Distância